

中山大學
SUN YAT-SEN UNIVERSITY



国家超级计算广州中心
NATIONAL SUPERCOMPUTER CENTER IN GUANGZHOU

计算机图形学

渲染管线

陶钧

taoj23@mail.sysu.edu.cn

中山大学 计算机学院

国家超级计算广州中心

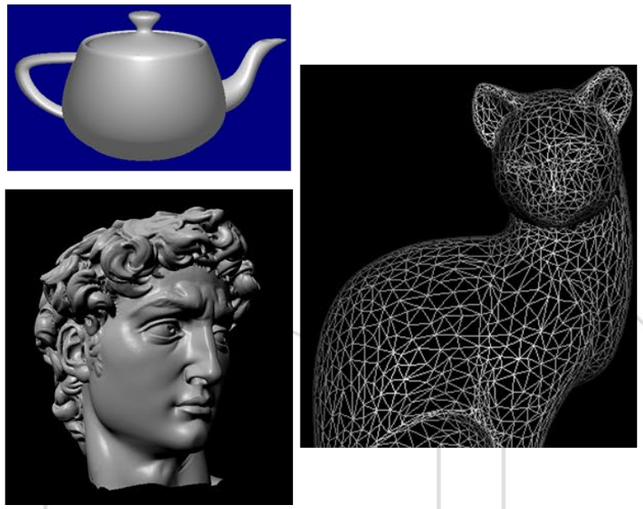
课程概要

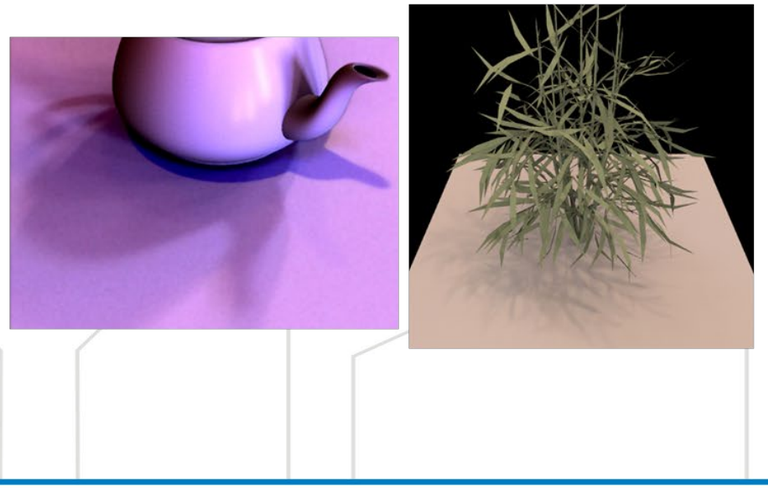
- 计算机图形系统
- 物理成像系统
- 图形渲染管线

Modeling
(Creating 3D Geometry)

→

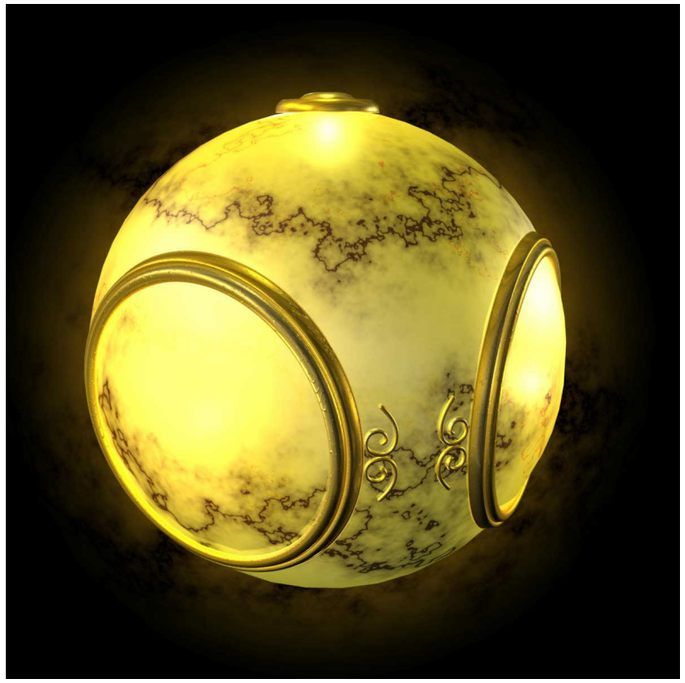
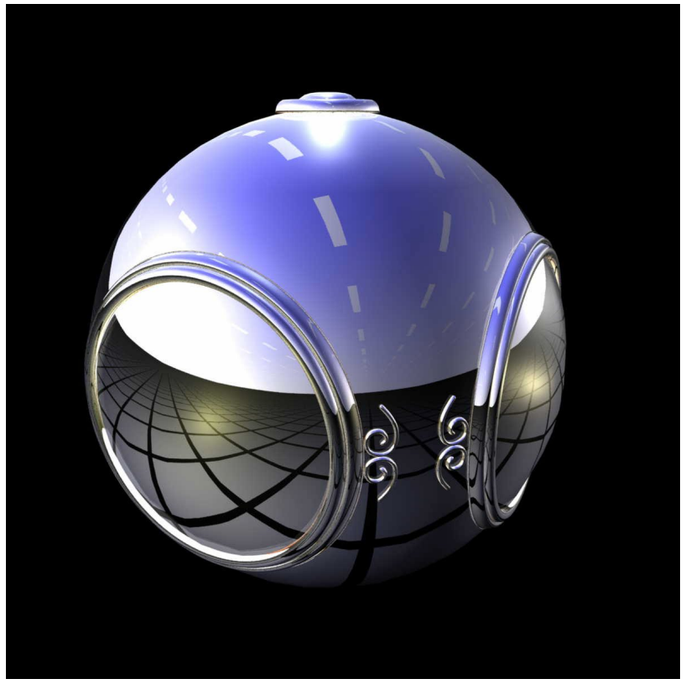
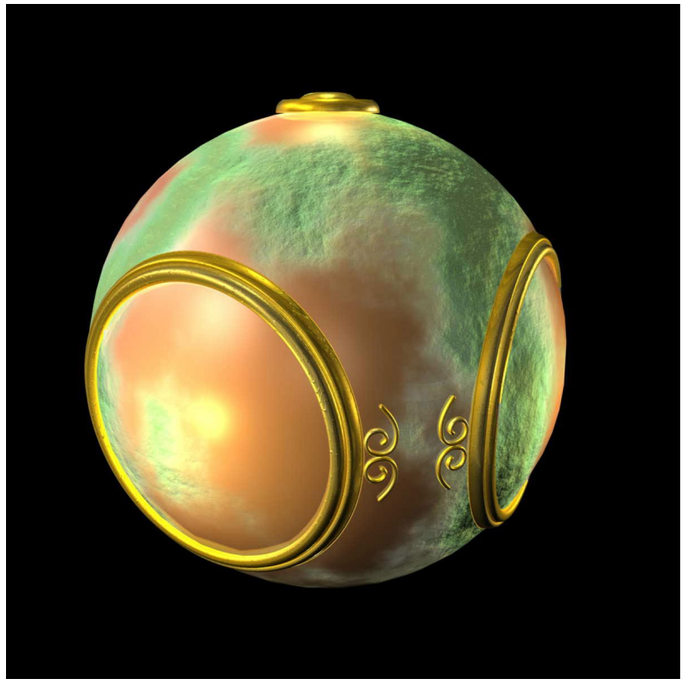
Rendering
(Creating, shading images from geometry, lighting, materials)





计算机图形系统

- 图像从何而来？

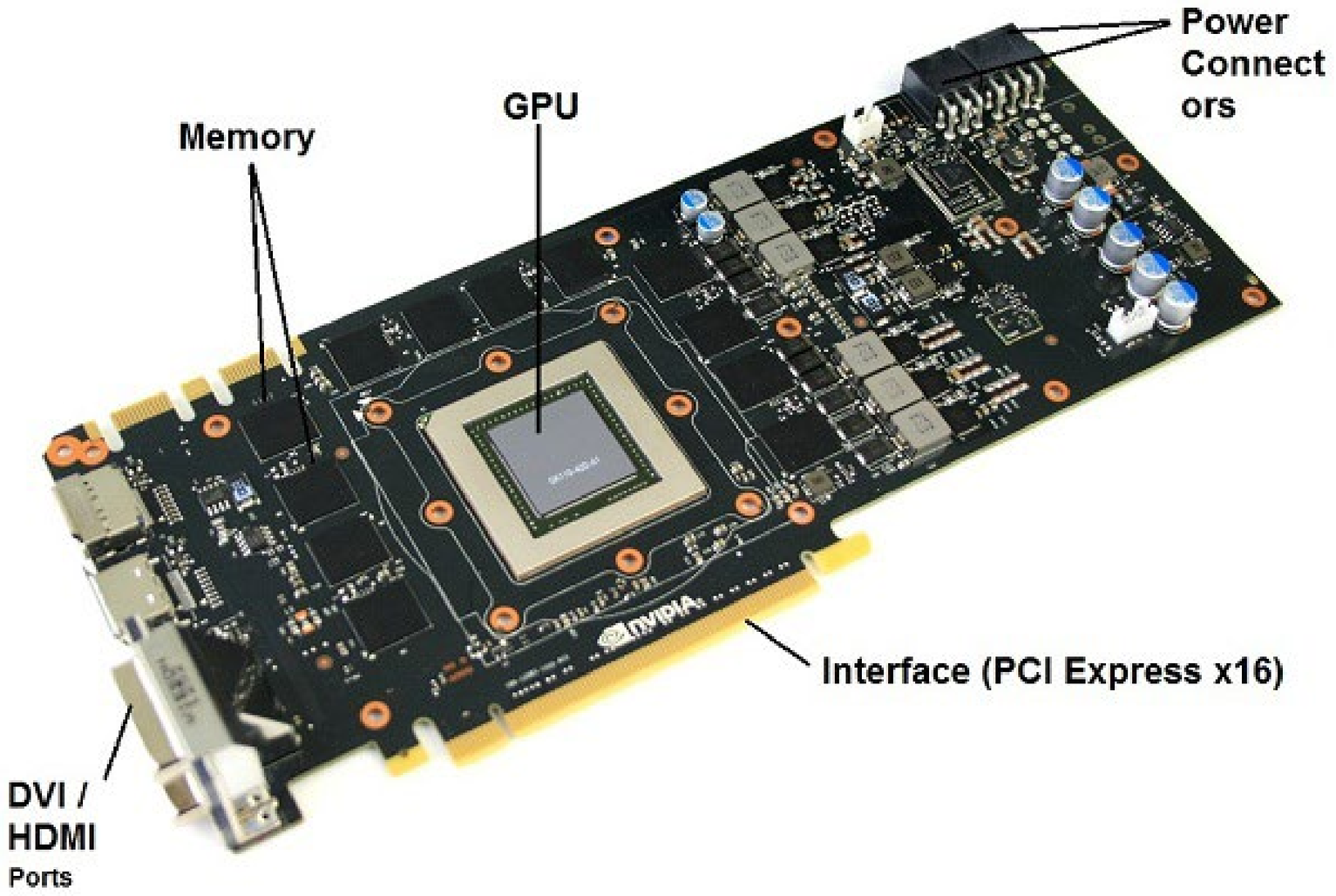
- 创建一张图像需要的软硬件？
  - 软件：建模及渲染软件（如Maya, RenderMan等）
  - 硬件：PC或大规模渲染集群上的显卡

- 软件 – Autodesk Maya
  - Happy buddha 模型



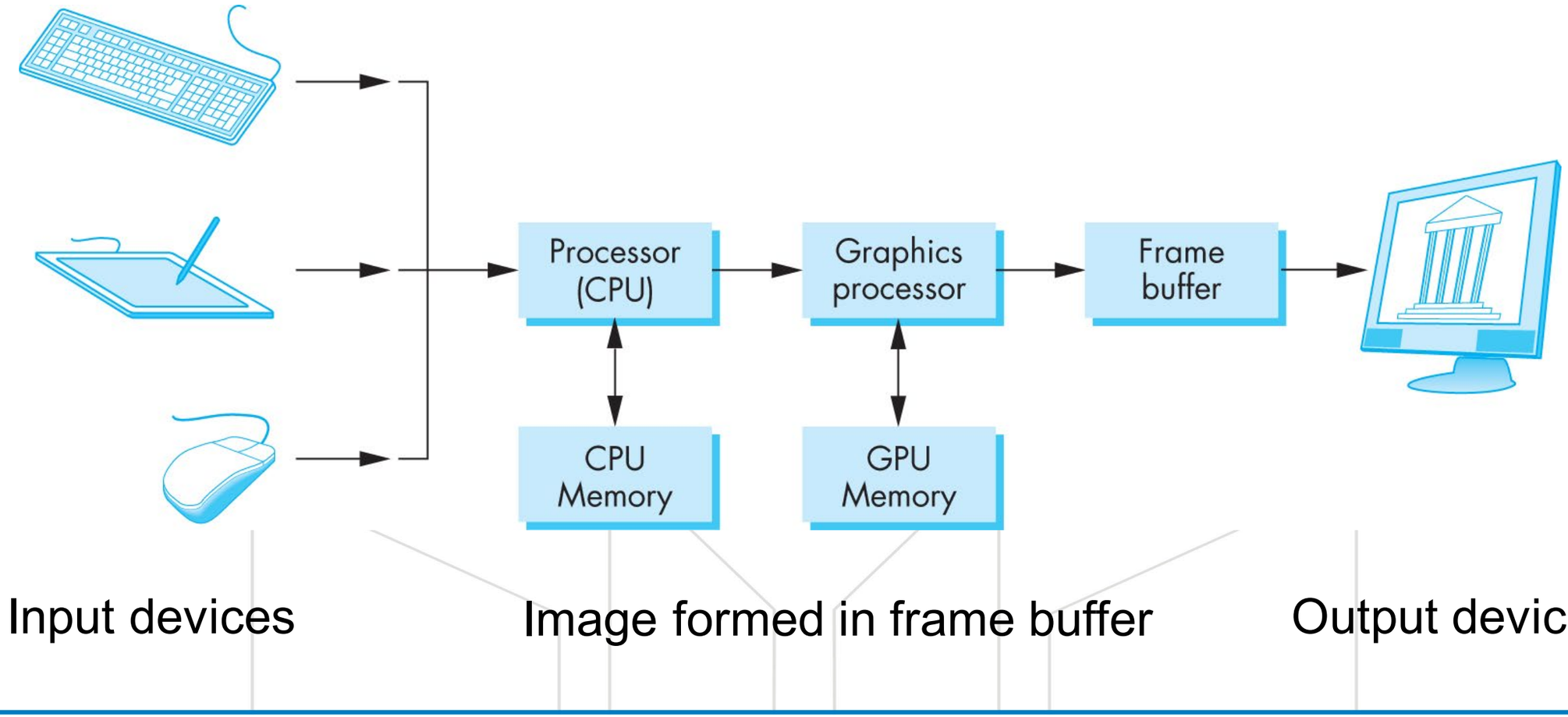
计算机图形系统

- 硬件 - 显卡



计算机图形系统

- 一个较为完整的计算机图形系统
  - 输入、处理设备、输出



- 输入设备：消费电子产品

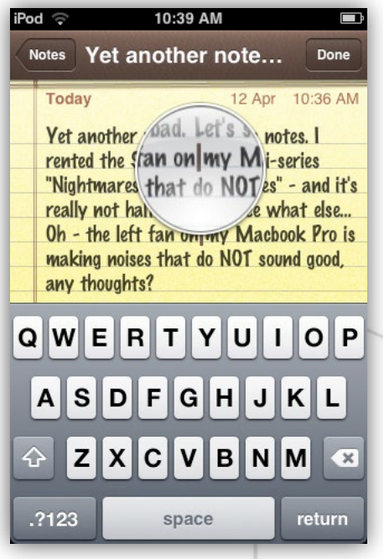
GRAPHIC|S

G|RAPHICS











- 输入设备：摄影摄像设备
  - 越来越触手可及





诺基亚N90（2005）
200万像素





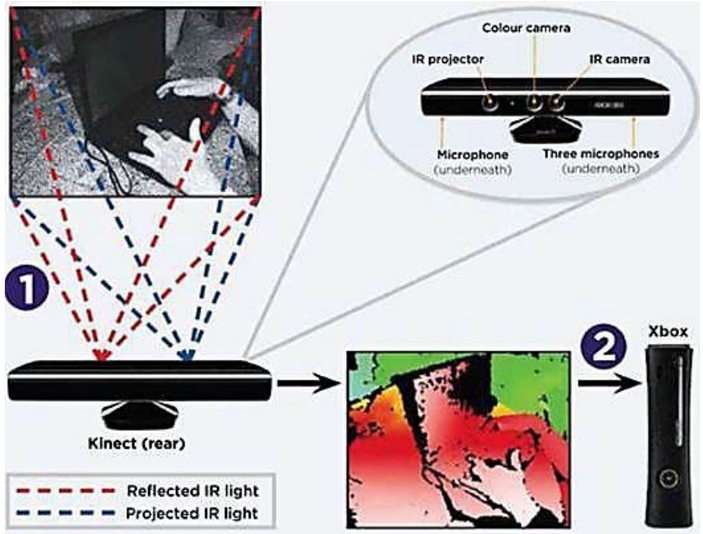
iPhone 6s Plus（2015）
前置：500万像素
后置：1200万像素

计算机图形系统

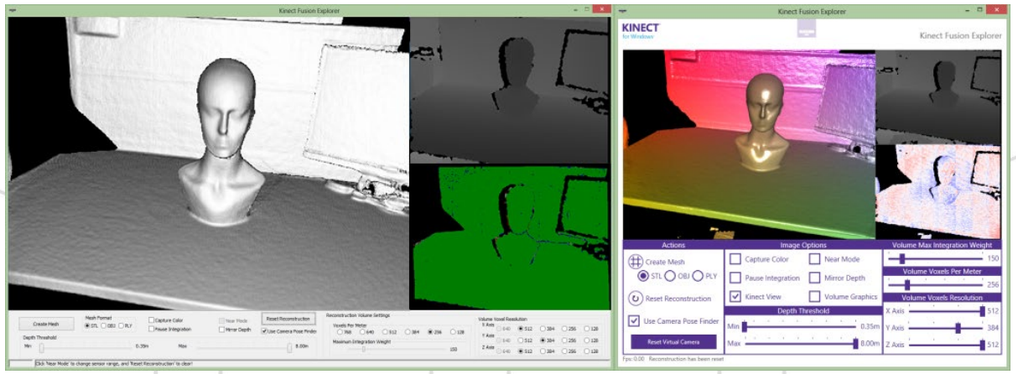
- 输入设备：几何体形状



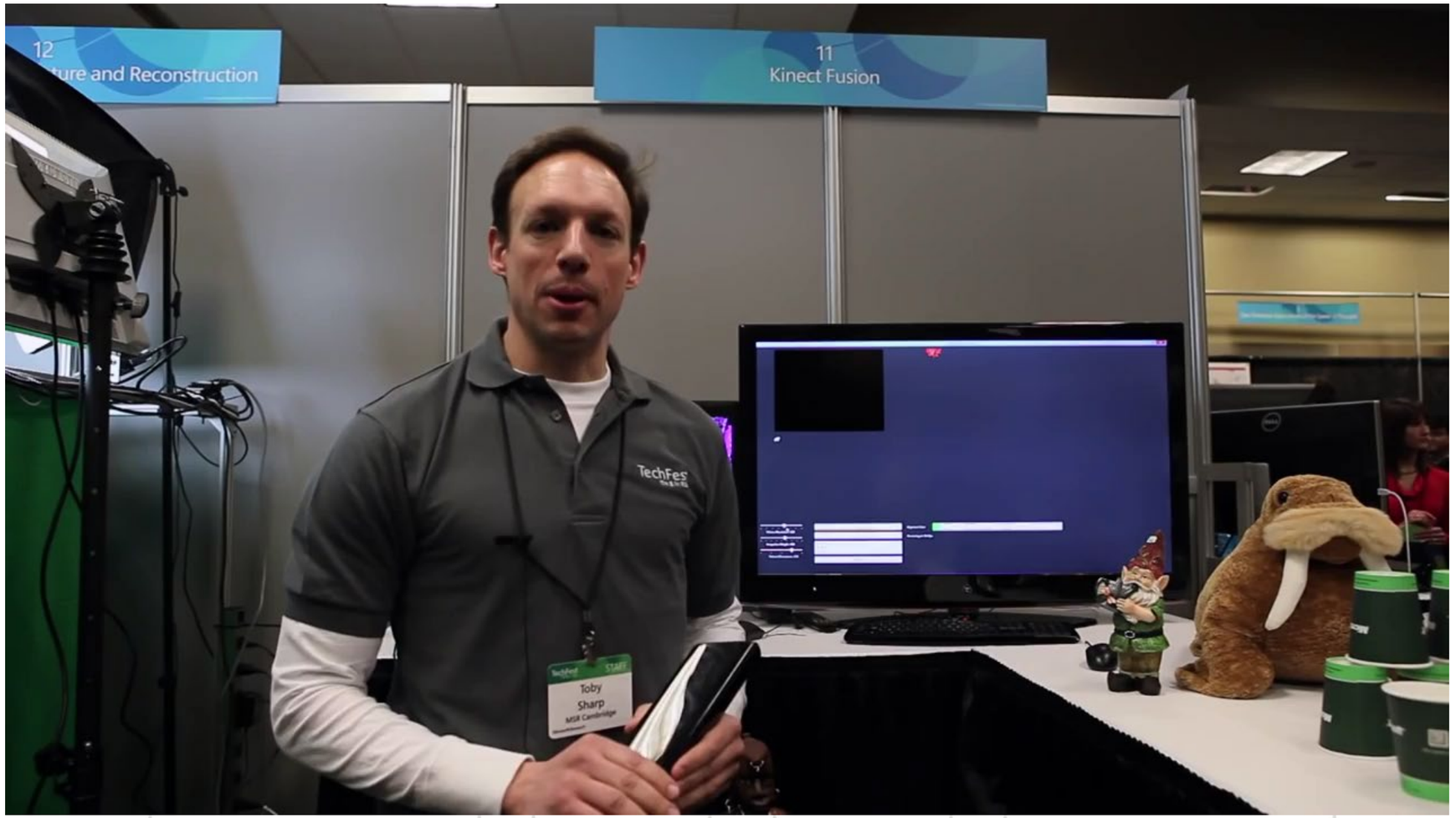
- 输入设备：几何体形状
  - 深度摄像头：Kinect



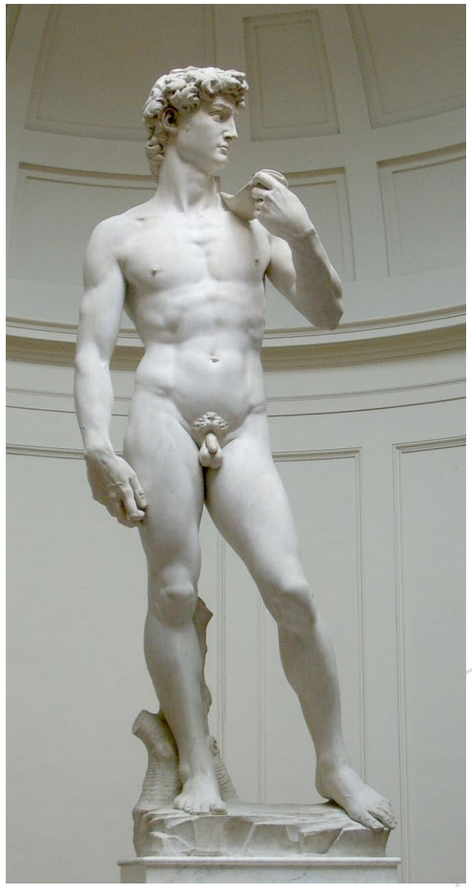


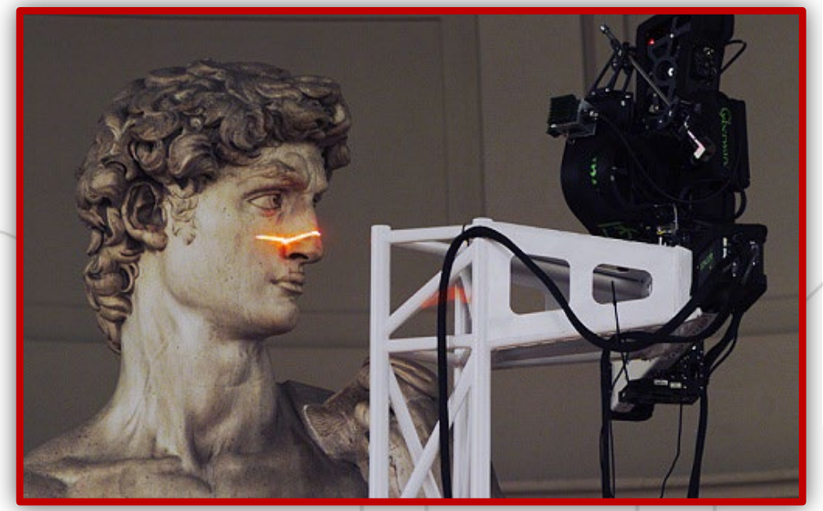


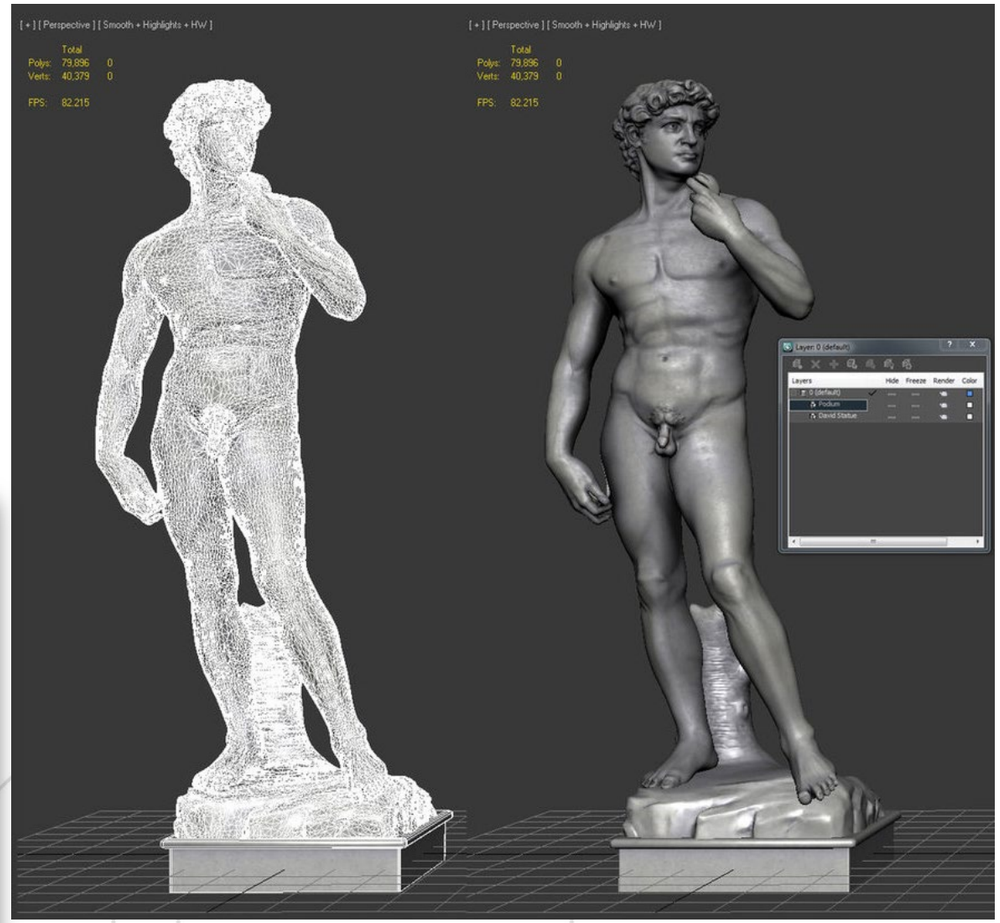
- 输入设备：几何体形状
  - 深度摄像头：Kinect



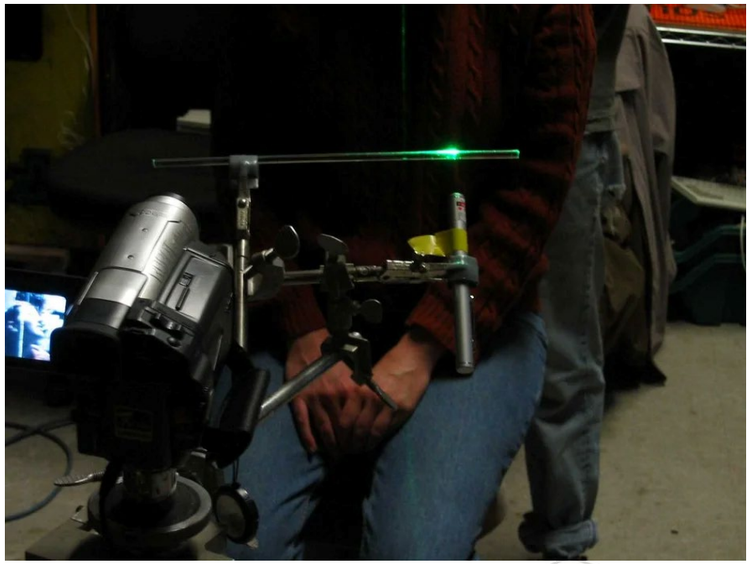
- 输入设备：几何体形状
  - 三维扫描





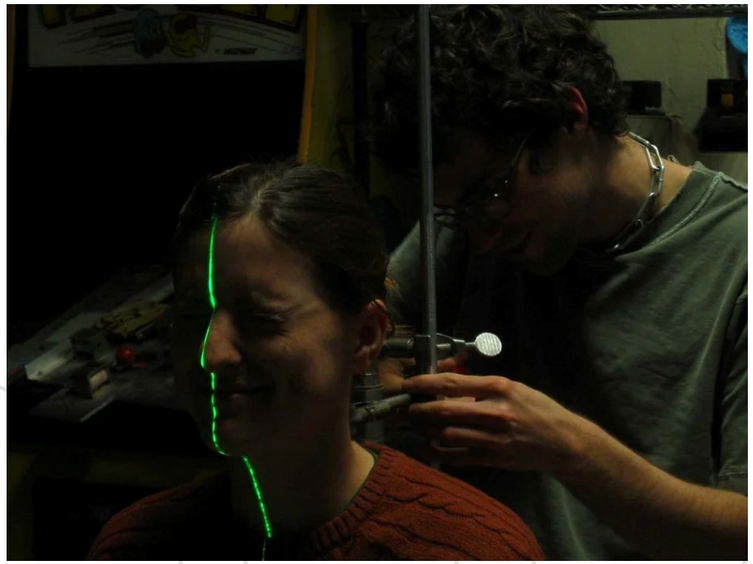


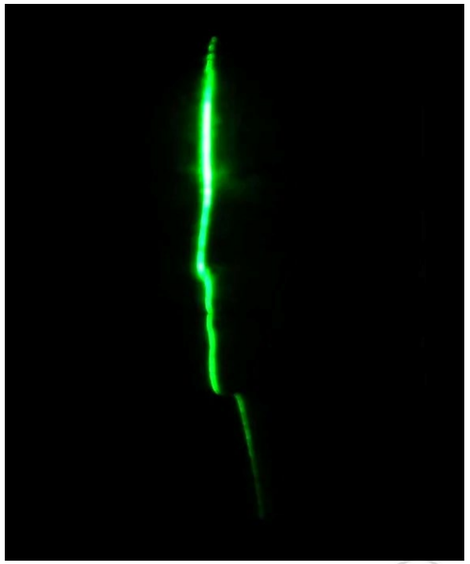
- 输入设备：几何体形状
  - 三维扫描：4步构建简易三维扫描系统！
    - https://www.instructables.com/id/3-D-Laser-Scanner/



设置摄像头、激光笔、圆柱透镜

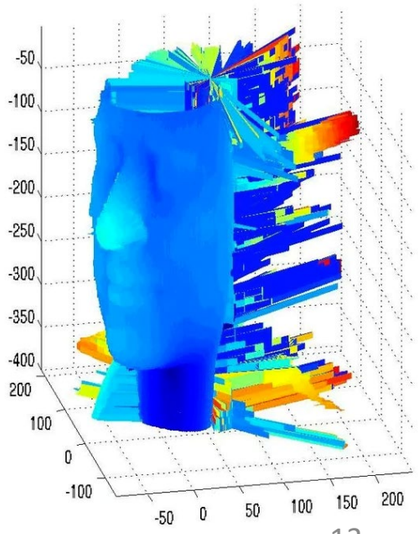
将扫描目标置于一个旋转平台上





捕获每个角度下激光扫描的图像并计算深度

从深度信息中还原三维模型



- 输入设备：几何体形状
  - 三维扫描：MakerBot



- 输入设备：其他专用设备







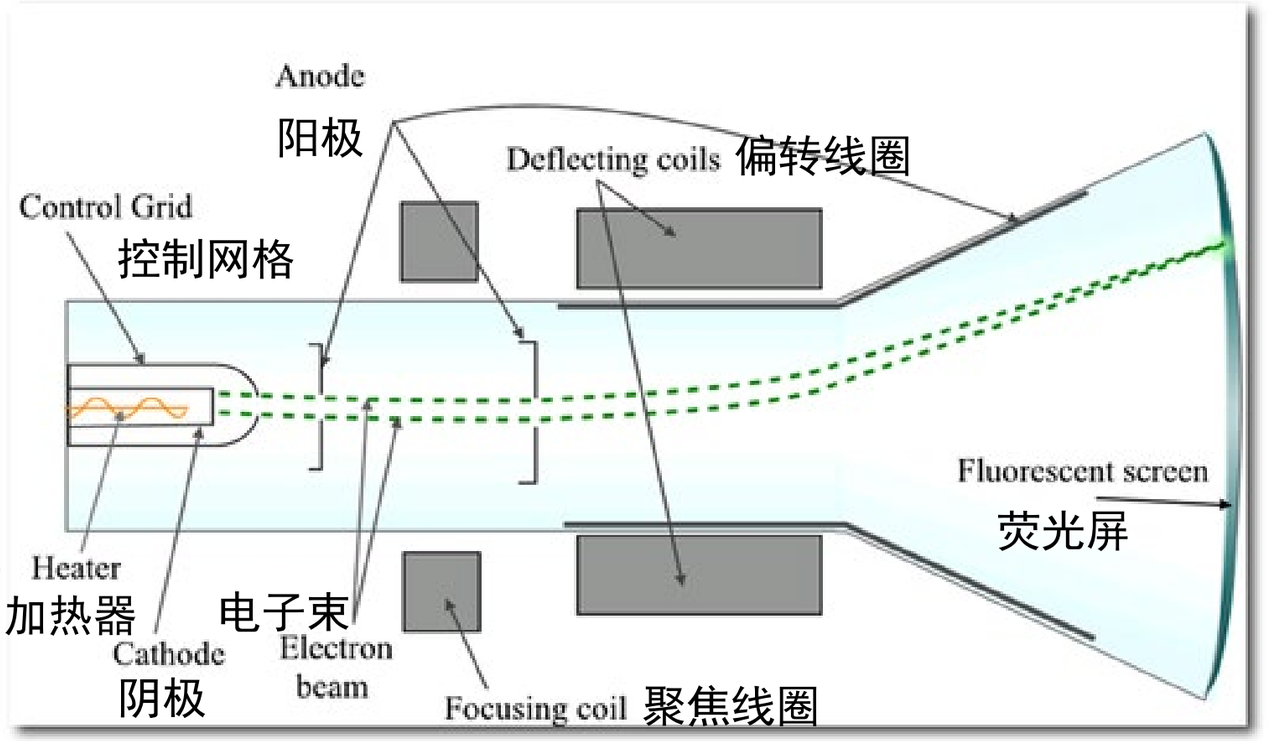
- 输出设备







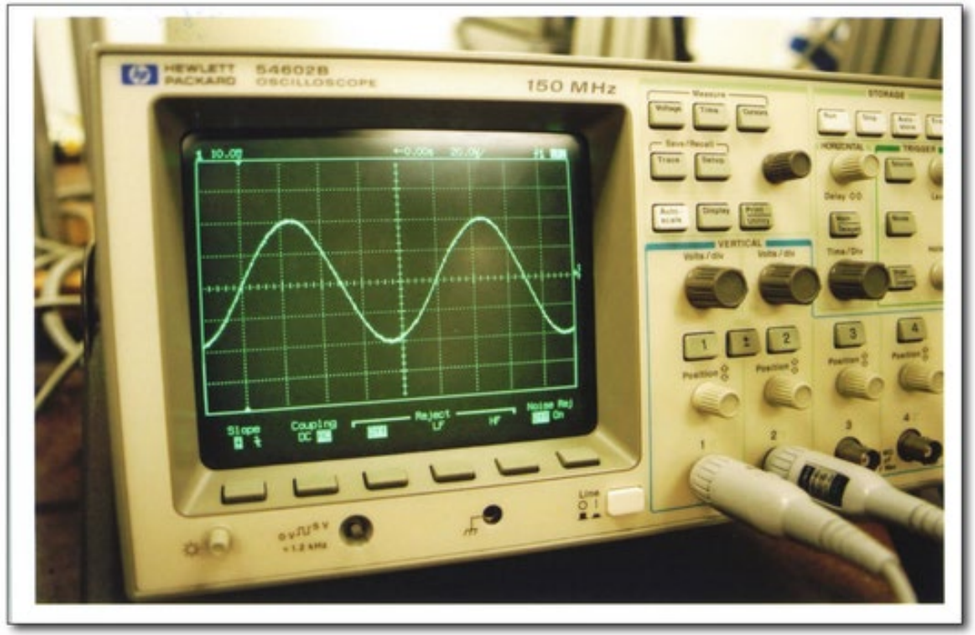
- 输出设备
  - 阴极射线管（Cathode Ray Tube，CRT）
    - 由电子枪发射出电子束（阴极射线）通过聚焦系统和偏转系统，射向屏幕上涂覆有荧光层的指定位置，从而产生图像。



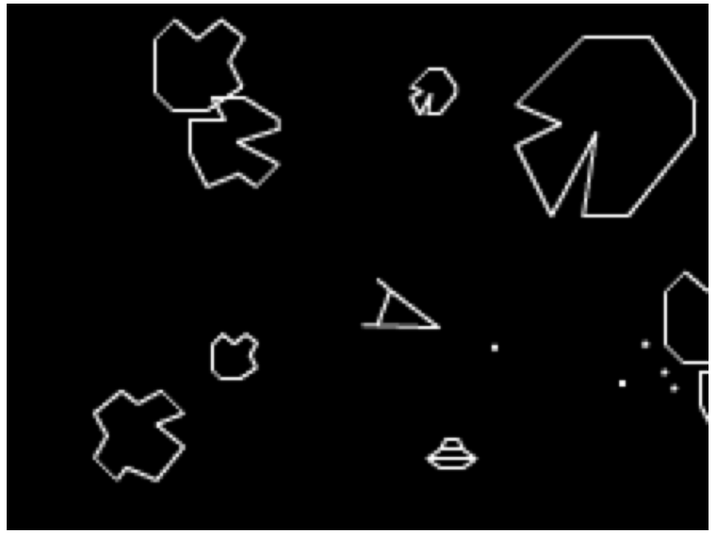
Karl Ferdinand Braun, 1897
图片来自Wikipedia

计算机图形系统

- 输出设备
  - CRT – 向量显示器（vector display）
    - 随机扫描显示器的一种
    - 沿任意路径画线，而非光栅化扫描



HP示波器

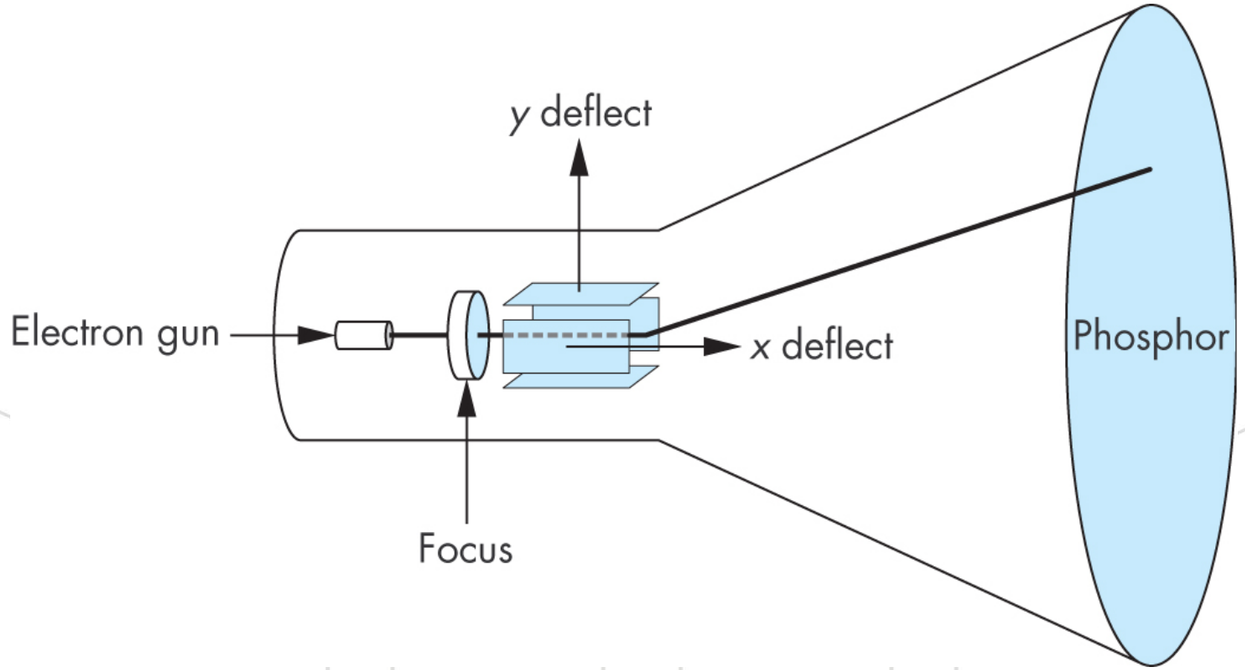


Asteroids, 1979 (行星游戏)

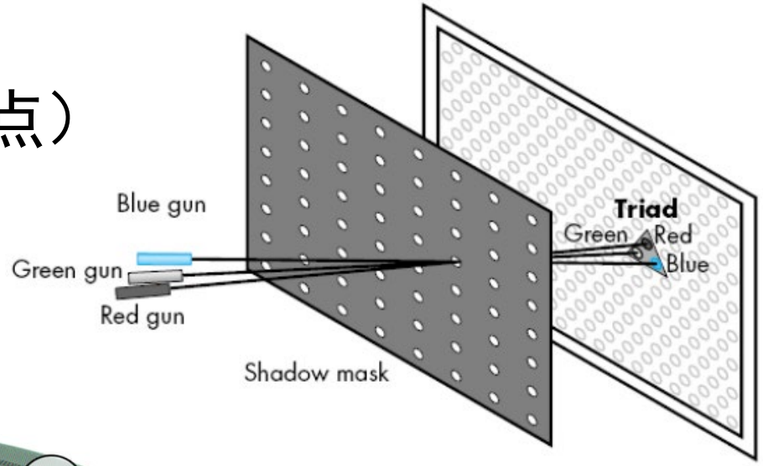


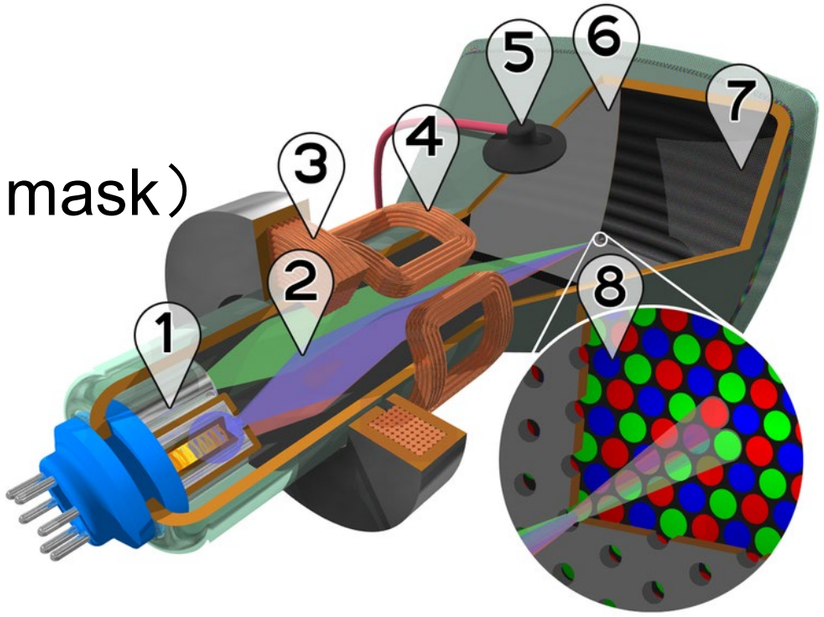
Star Wars, 1983 (星球大战)

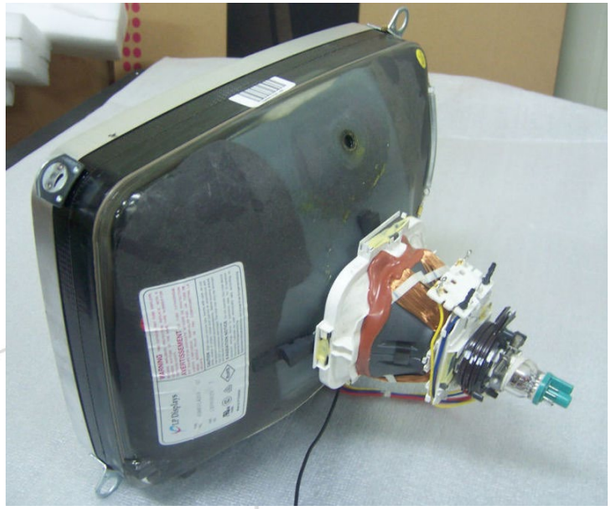
- 输出设备
  - CRT – 光栅显示器（raster display）
    - 刷新式光栅扫描显示：将frame buffer中的像素显示于屏幕上的相应位置
    - 刷新率（refresh rate）
    - 模式：逐行扫描、隔行扫描（主要用于刷新率较低的设备）



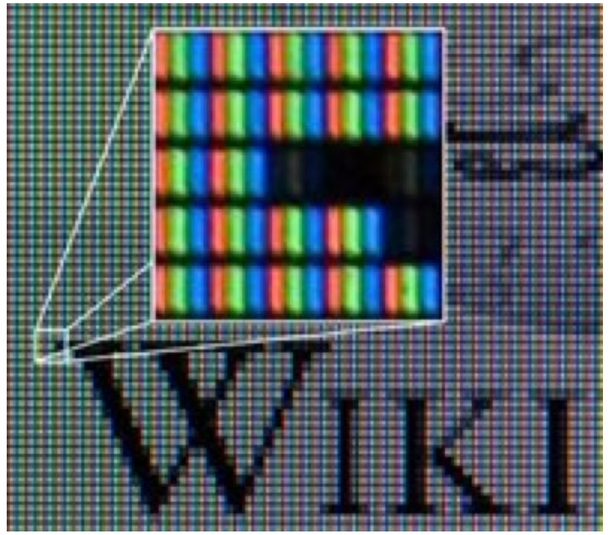
- 输出设备
  - 彩色CRT显示器
    - 1. 三把电子枪（分别射向红绿蓝三个荧光点）
    - 2. 电子束
    - 3. 聚焦线圈
    - 4. 偏转线圈
    - 5. 阳极
    - 6. 荫罩（shadow mask）
    - 7. 荧光层
    - 8. 荧光层细节



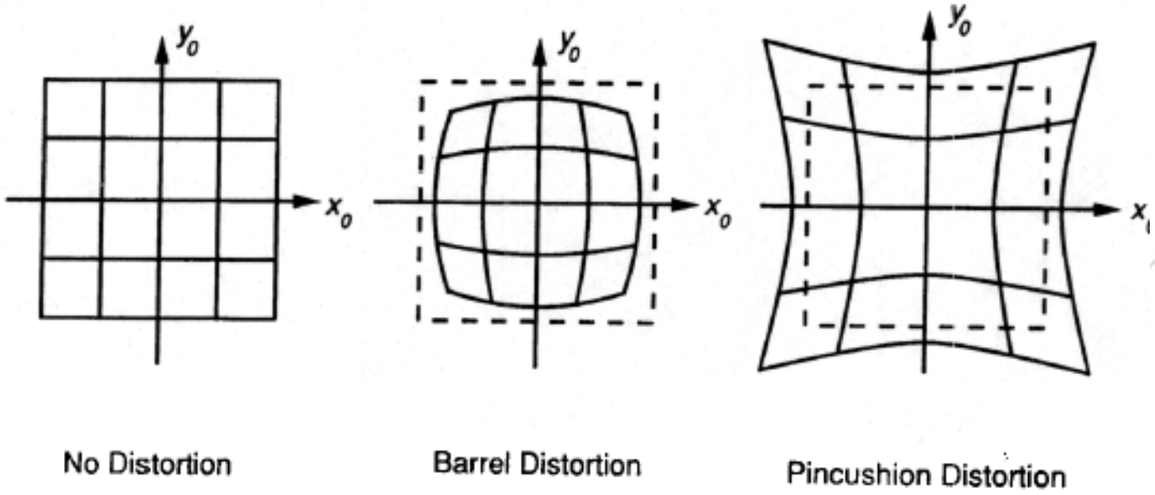


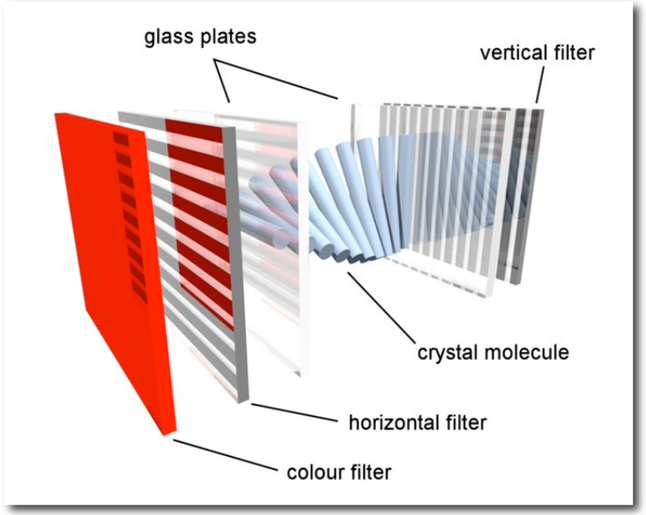


- 输出设备
  - 平板显示器（flat-panel monitors）
    - Plasma panels：等离子显示器
    - LCD（liquid-crystal display）：液晶显示器
    - LED（light-emitting diodes）：发光二极管
    - 优点：低能耗，低辐射，无闪烁，无几何失真



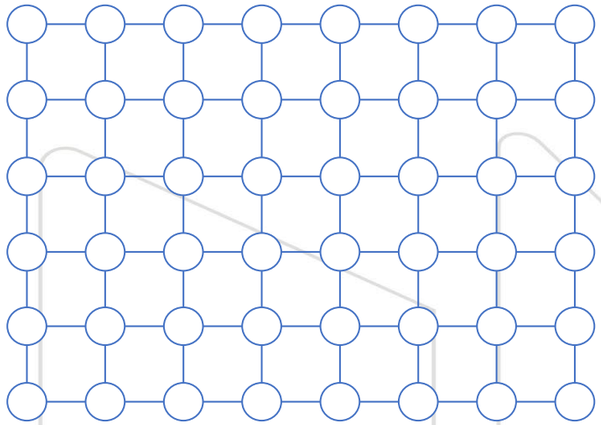
显示彩色时同样需要通过三个亚像素

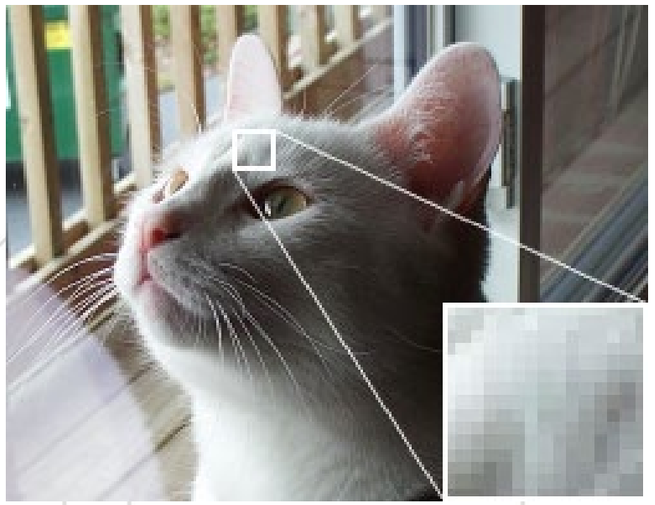




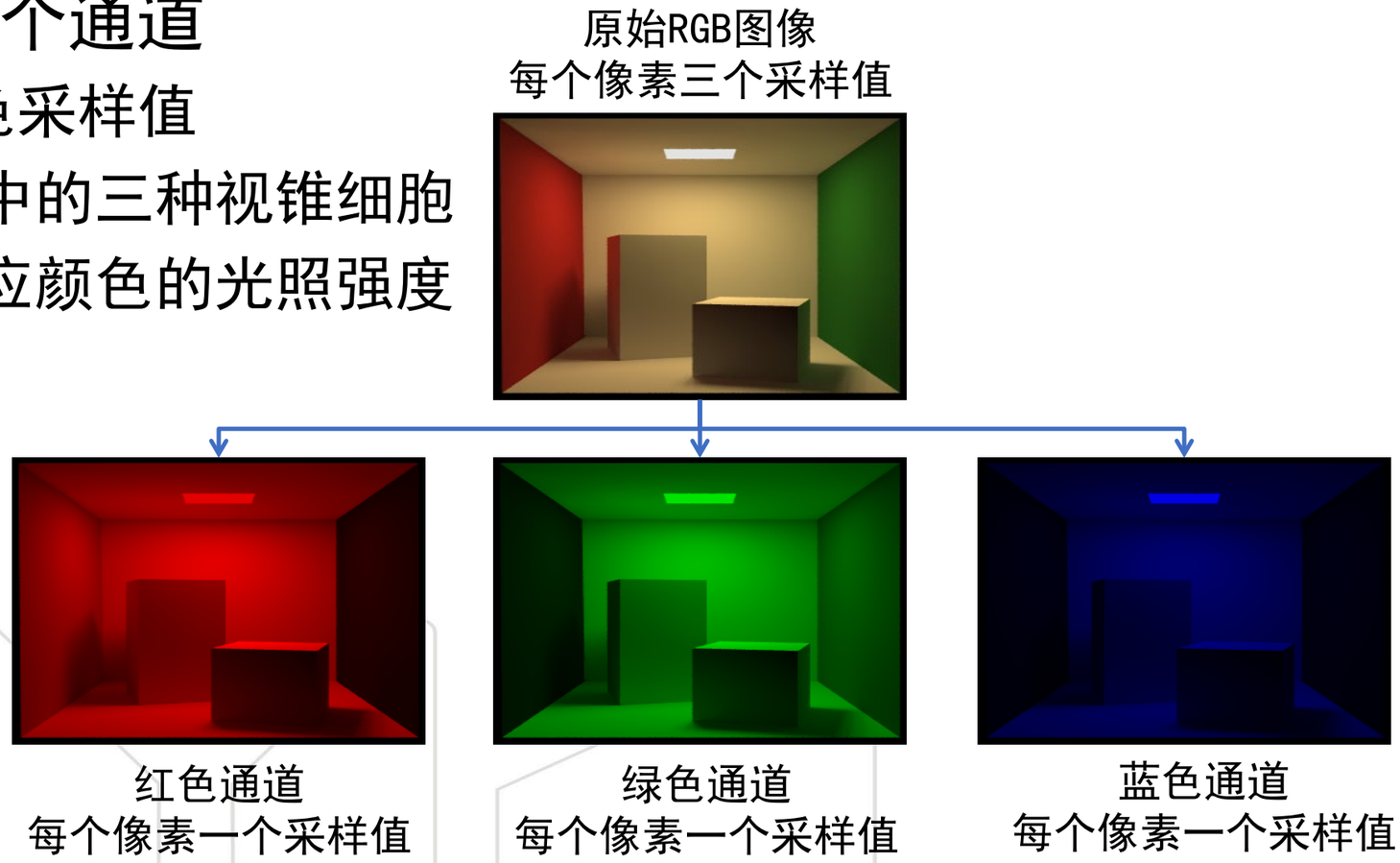
单个亚像素

- 图像
  - 有规律采样的二维域（domain）
    - 几乎都是二维直线网格
    - 每个采样点可以有多个值（如颜色RGB）
    - 采样值的含义由应用决定（RGBA，深度，等）
    - 单位同样取决于应用
      - 整型、浮点型数值可以被映射至电压从而显示在屏幕上
      - 光照强度可由摄像机感光元件转为图像





- 图像
  - 一个通道是所有统一类型的采样值
  - RGB是图像中最常见的三个通道
    - 如，“R”通道只包含红色采样值
    - 大致对应于人类视觉系统中的三种视锥细胞
    - 每个采样值对应于该点相应颜色的光照强度

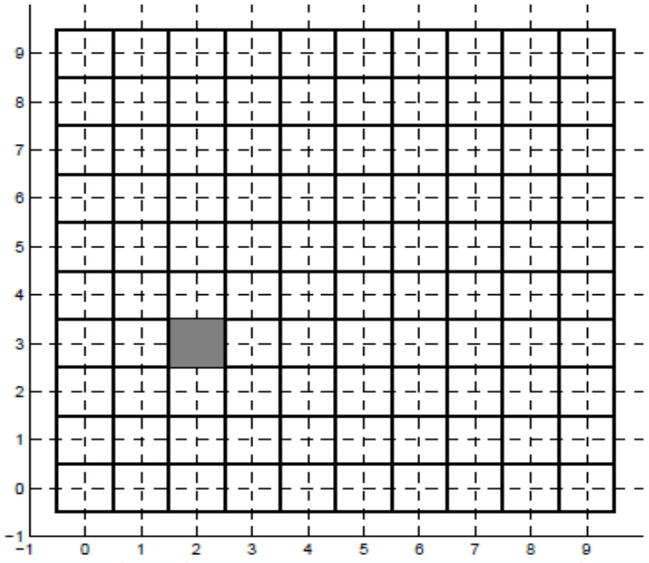


- 图像
  - Alpha通道
    - RGB外最常见的第四个通道，用于表示透明度
      - transparency/opacity/translucency
    - 取值在 [0, 1] 之间
      - “1”表示完全不透明
      - “0”表示完全透明
    - 可用于图像的颜色混合
      - 高alpha值表示低透明度
      - 使用Porter-Duff合成（alpha合成）
      - $\alpha X + (1 - \alpha)Y$

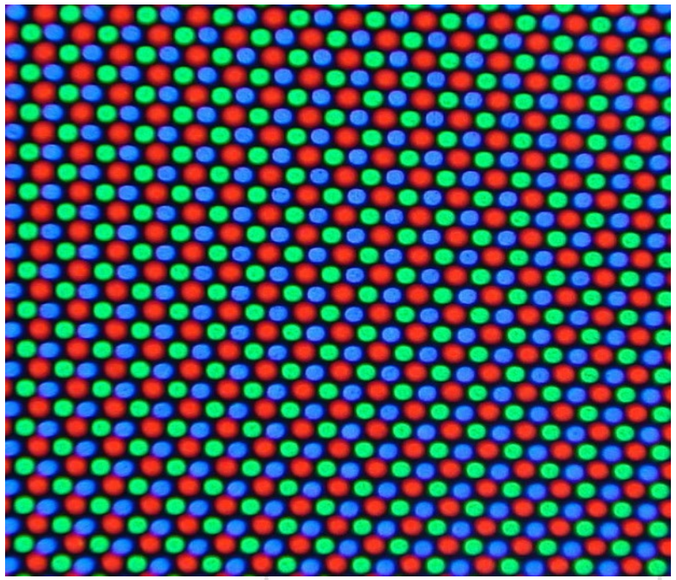


$\alpha = 0.8$的橙色长方形绘制于紫色长方形之上的结果

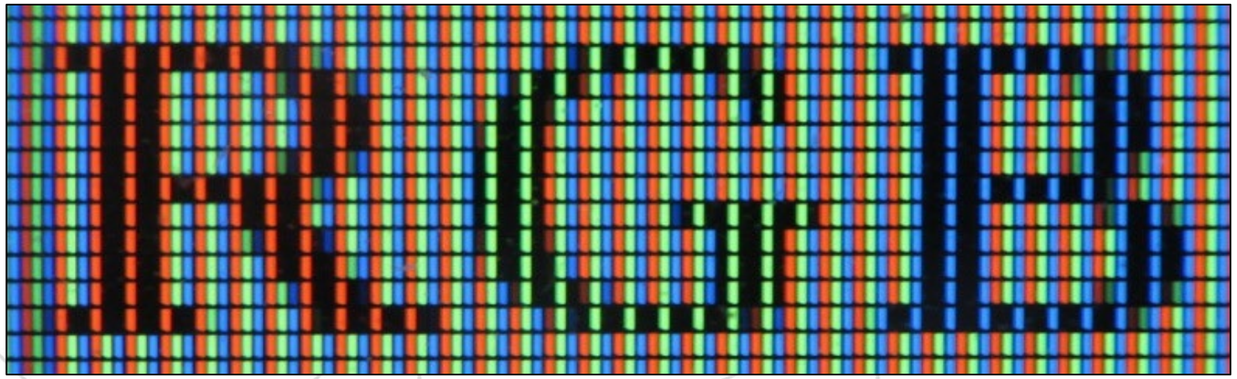
- 图像建模
  - 一张$m \times n$的图像可表示为一个函数$u(i,j)$
    - 将一对整数（像素坐标）映射至实数
    - $0 \le i < m, 0 \le j < n$为整数
  - 每一个像素值对应于显示器相应坐标上的一个小区域
  - 像素通常被想象为以采样点为中心的一个正方形，但其具体形状往往依赖于显示设备
    - 在CRT显示器接近圆形
    - 在LCD面板上接近于长方形



- 图像建模
  - 像素准确地说为采样点+值（而非方块）
  - 显示器往往使用多个亚像素在屏幕上重构“像素”
    - 每个亚像素对应一个原色

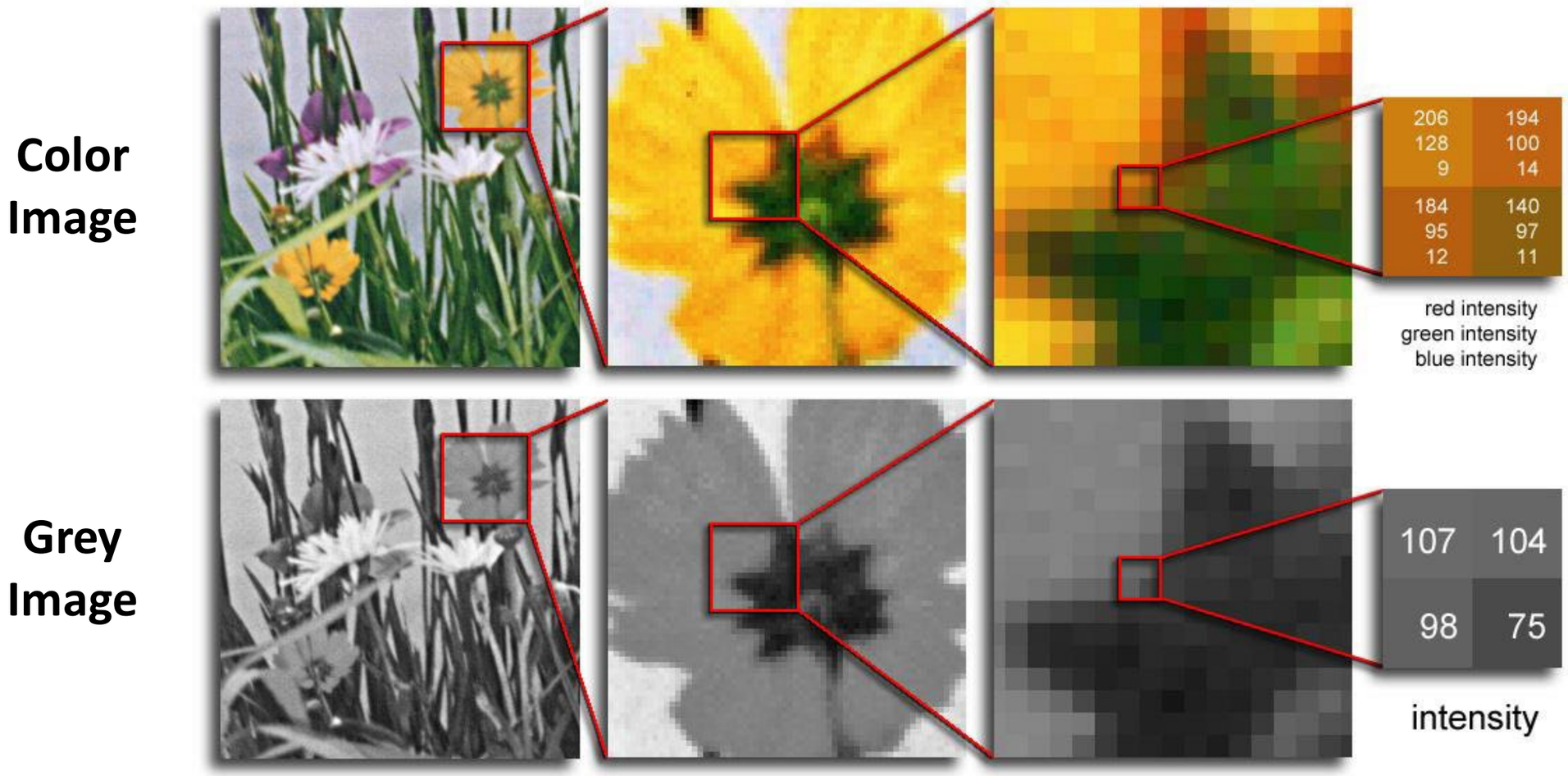


Close-up of a CRT screen



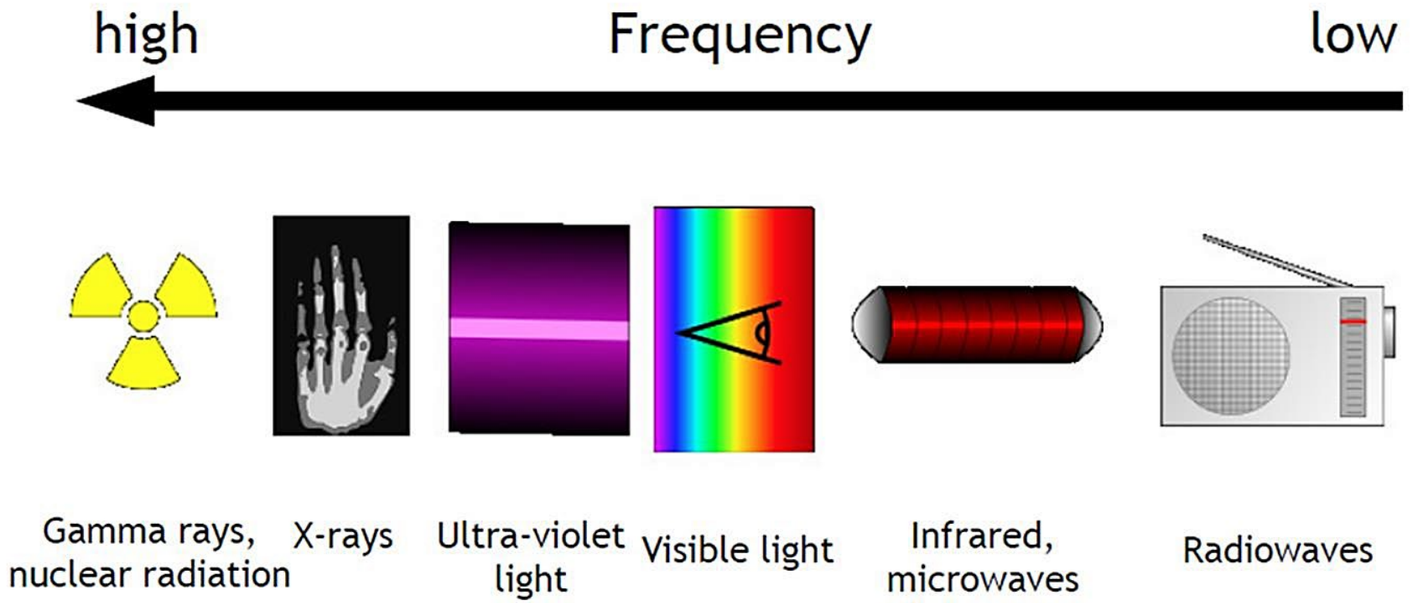
Close-up of an LCD screen

- 数码图像


Color Image
206 194
128 100
9 14
184 140
95 97
12 11
red intensity
green intensity
blue intensity
Grey Image
107 104
98 75
intensity

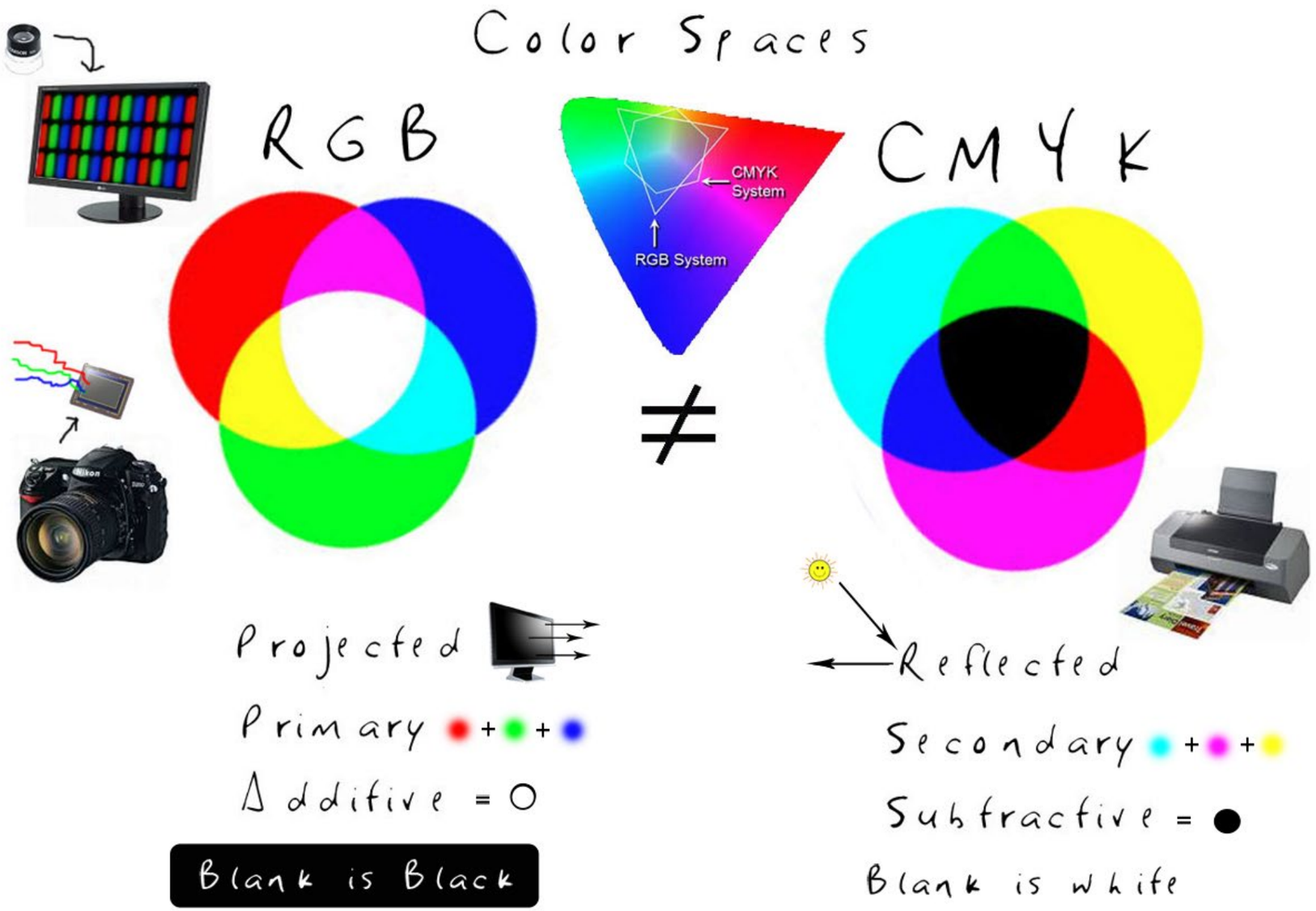
- 可见光
  - 光的本质是电磁波
  - 可见光的范围：350nm至780nm之间

Different frequencies



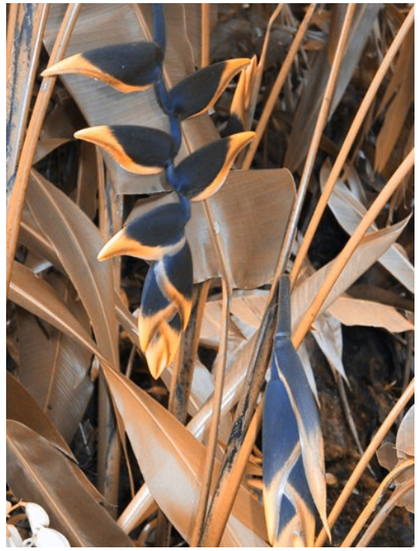
计算机图形系统

- 颜色空间

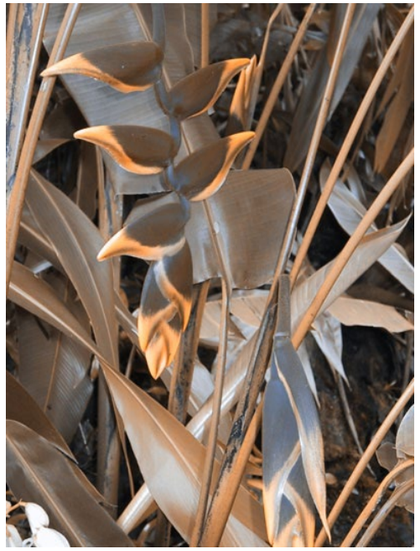


色盲





红色盲
（Protanope）
无L锥形细胞



绿色盲
（Deuteranope）
无M锥形细胞



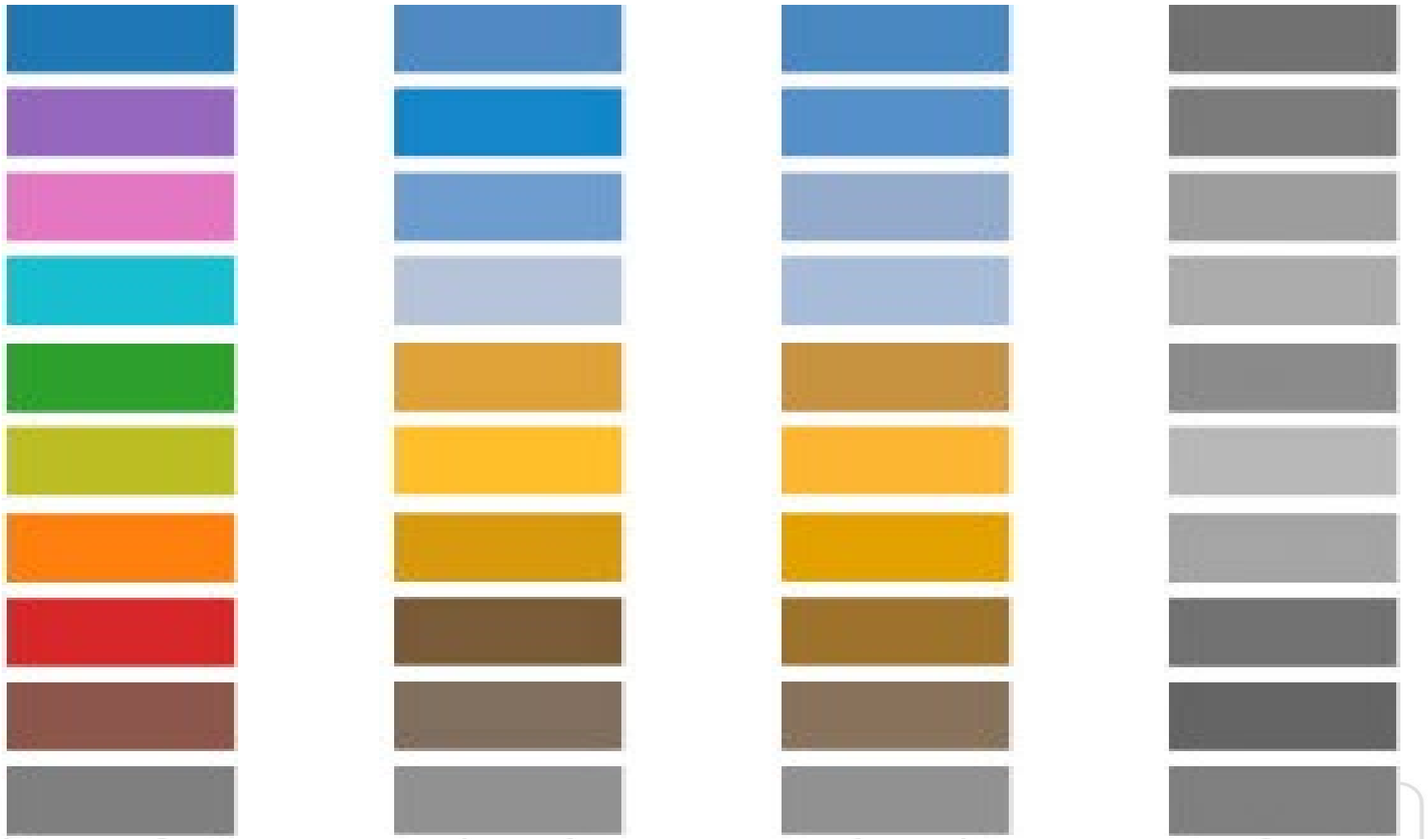
蓝色盲
（Tritanope）
无S锥形细胞

红/绿色盲

蓝/黄色盲

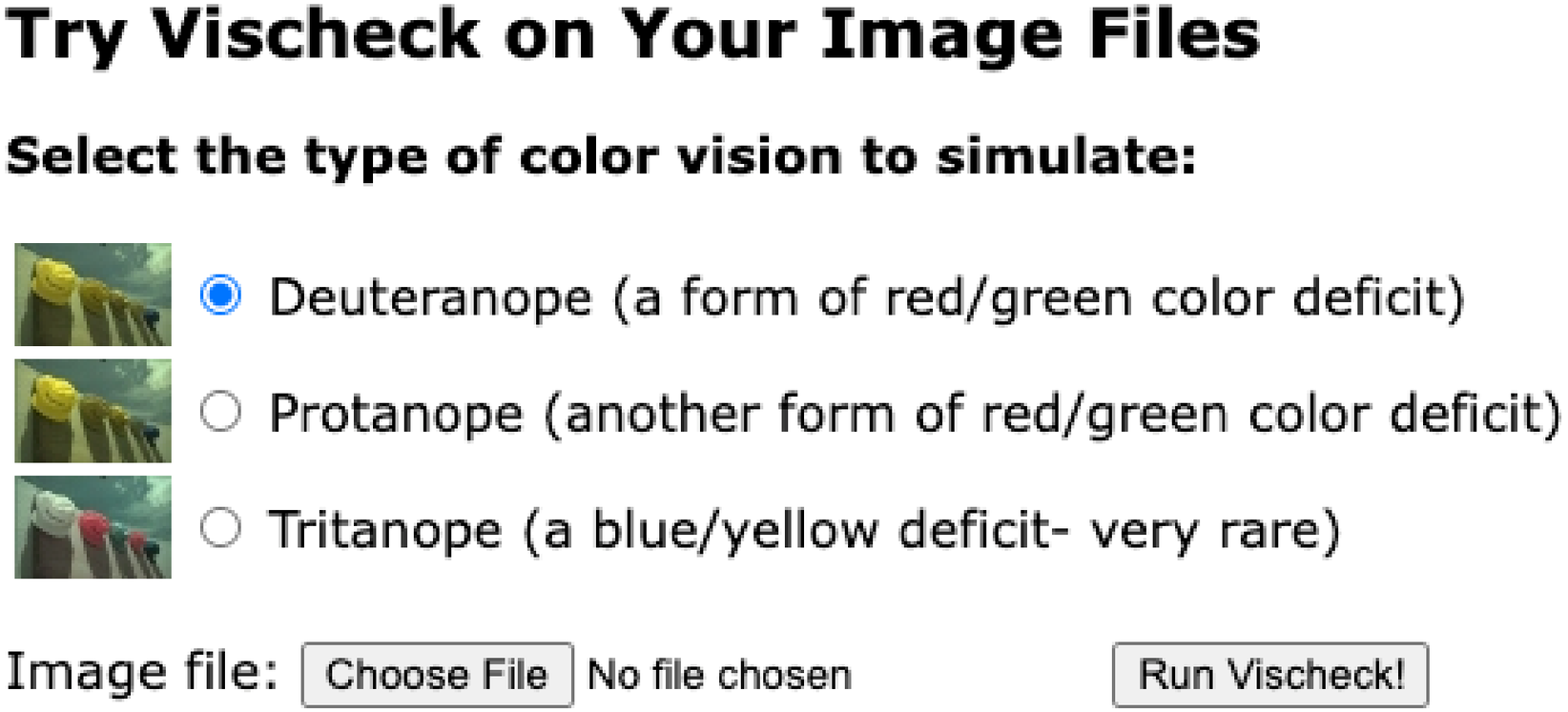
Based on slide from Stone

- 色盲



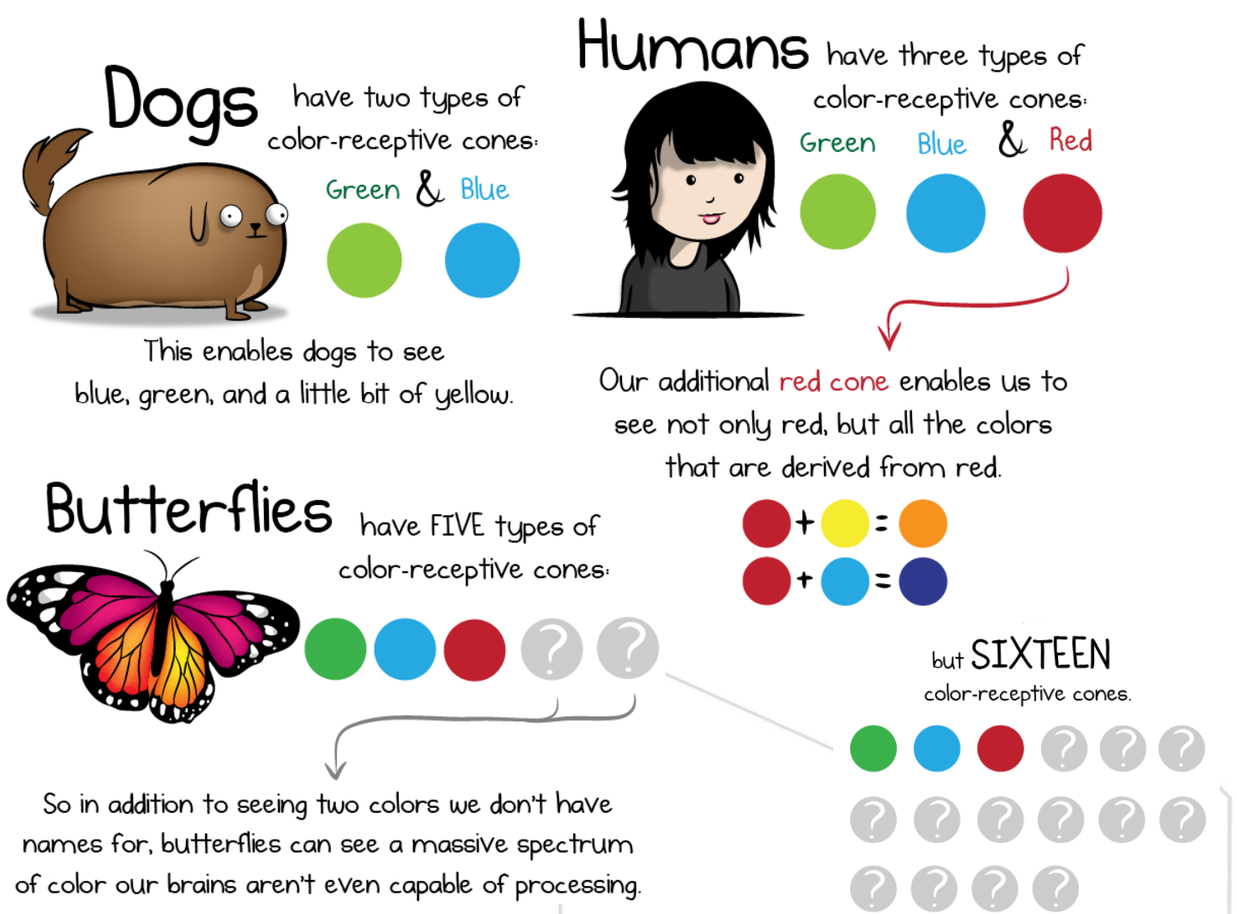
Based on slide from Stone

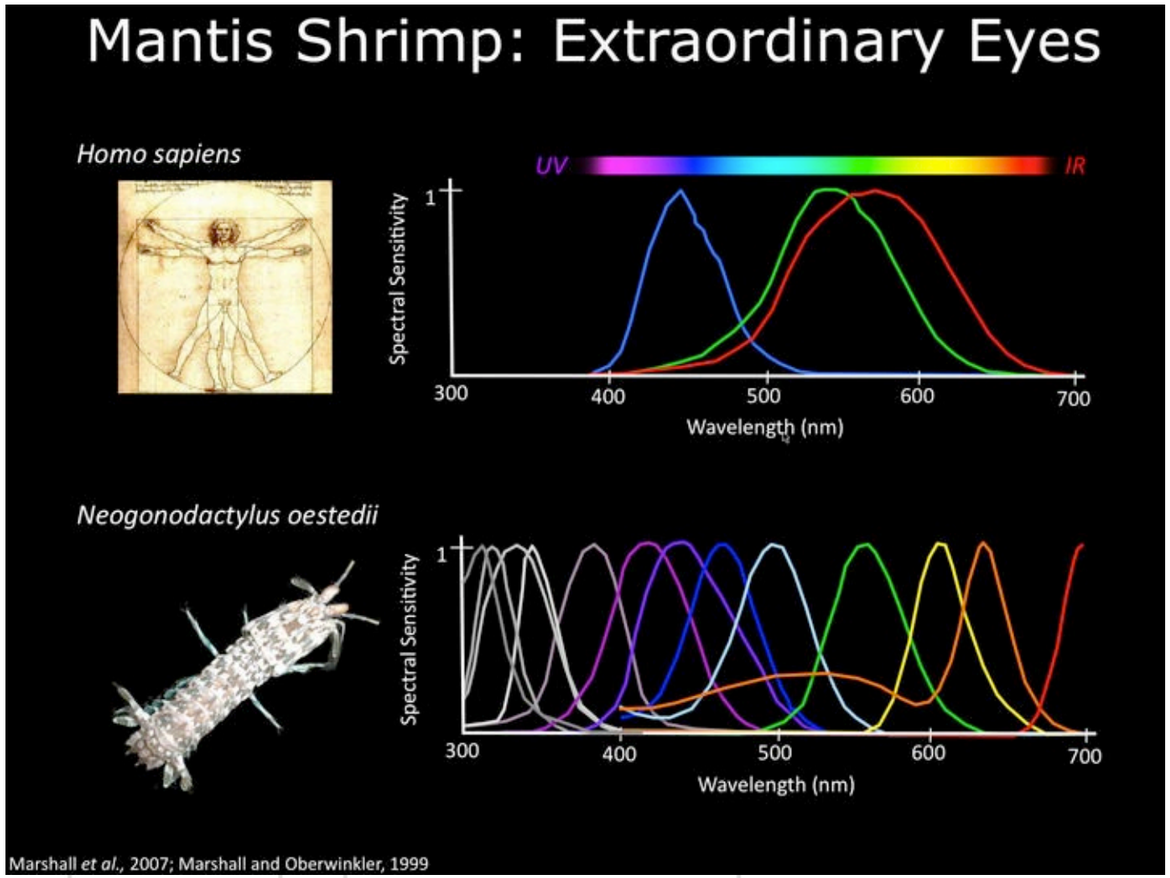
- 色盲
  - VisCheck.com
    - 色盲模拟器
    - 上传图片
    - 输入网址



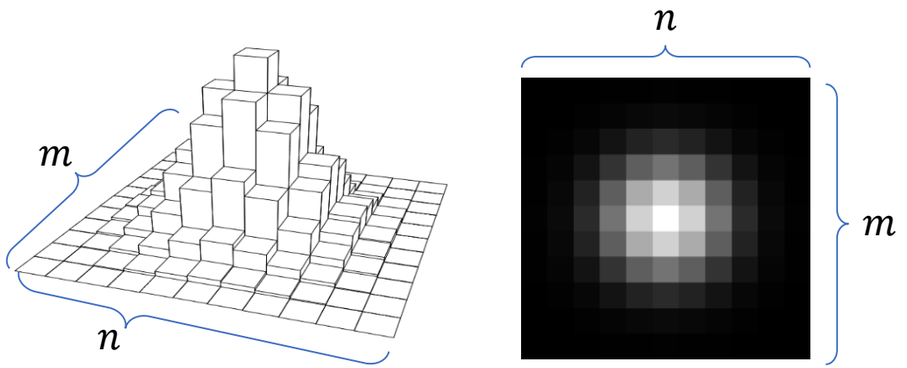
动物界色锥细胞数量

- 据说发现了四色视者：https://jov.arvojournals.org/article.aspx?articleid=2191517
- Why the mantis shrimp is my new favorite animal: https://theoatmeal.com/comics/mantis_shrimp

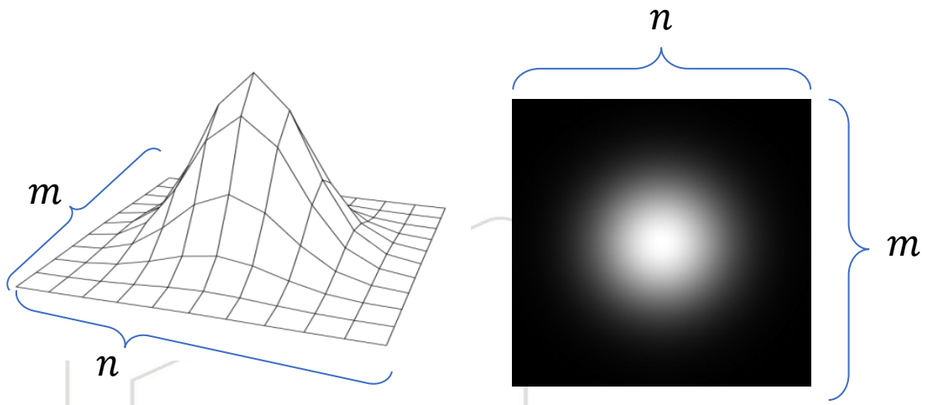




- 图像建模
  - 离散图像
    - $\mathbb{Z}^2 \to \mathbb{R}$
    - 图像最为常见的存储形式
  - 连续图像
    - $\mathbb{R}^2 \to \mathbb{R}$
    - 真实世界中最常见的表现形式
    - 连续指定义域（domain）连续
      - 而非其取值连续

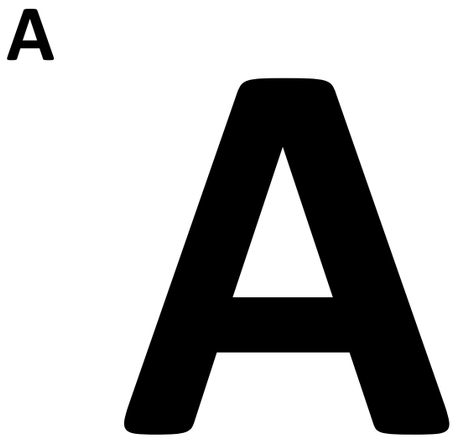


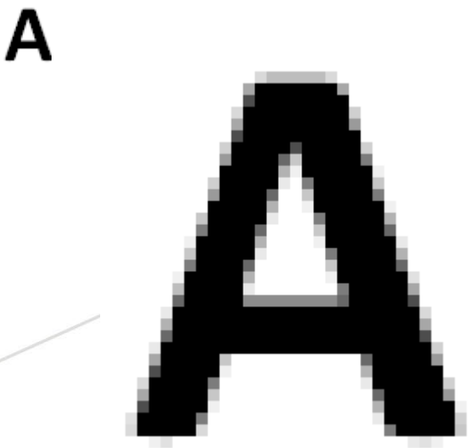
离散高斯核



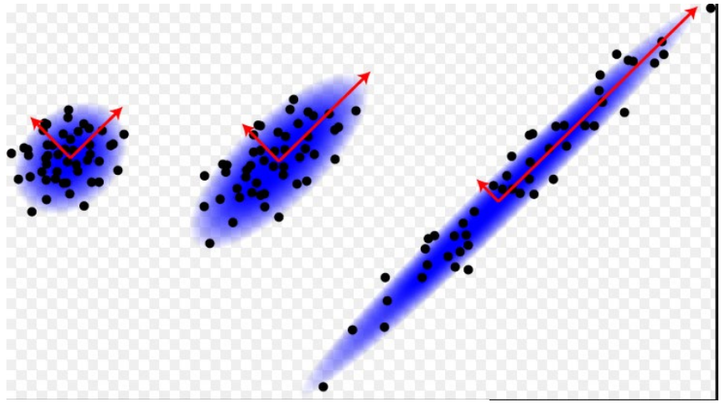
连续高斯核

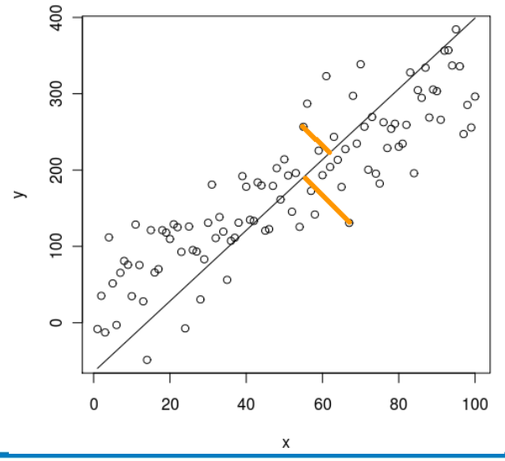
- 图像建模
  - 矢量图
    - 随机扫描中常用
    - 由绘制命令驱动
      - 如，move(x, y), char(“A”), line (x, y)，等
    - 可缩放矢量图形
  - 光栅化图像
    - 电视、打印机中常用表现形式
    - 像素驱动
    - 由于离散采样可能产生锯齿





- 如何理解图像？
  - http://staff.ustc.edu.cn/~lgliu/Courses/ComputerGraphics_2018_spring-summer/default.htm
  - 图像是数据点的集合
    - 考虑每个像素为RGB空间中的一个点，则计算灰度图像成为降维问题







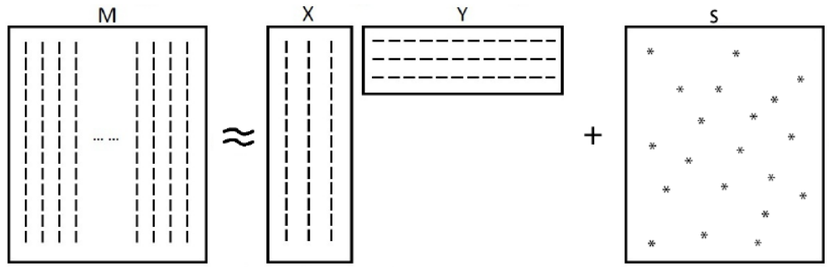


直接转换

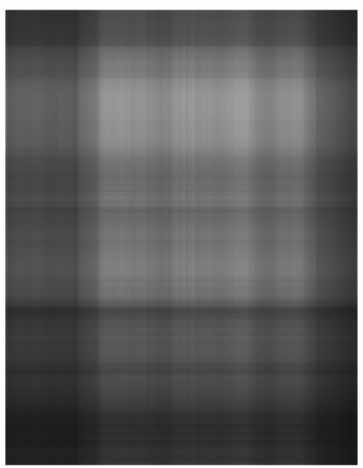


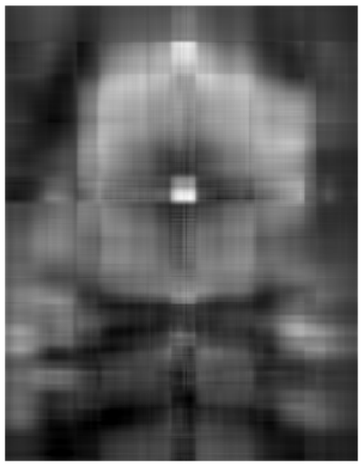
降维

- 如何理解图像？
  - 图像是一个矩阵
    - 矩阵分解、低秩矩阵等方法可以应用
    - 图像的低秩近似



M = low-rank matrix XY + sparse matrix S
(robust PCA)


Rank 1


Rank 5

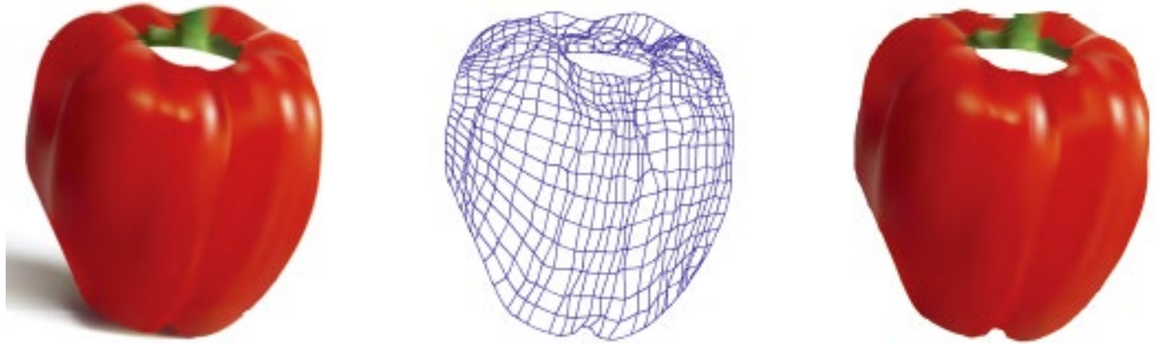

Rank 20


Rank 30

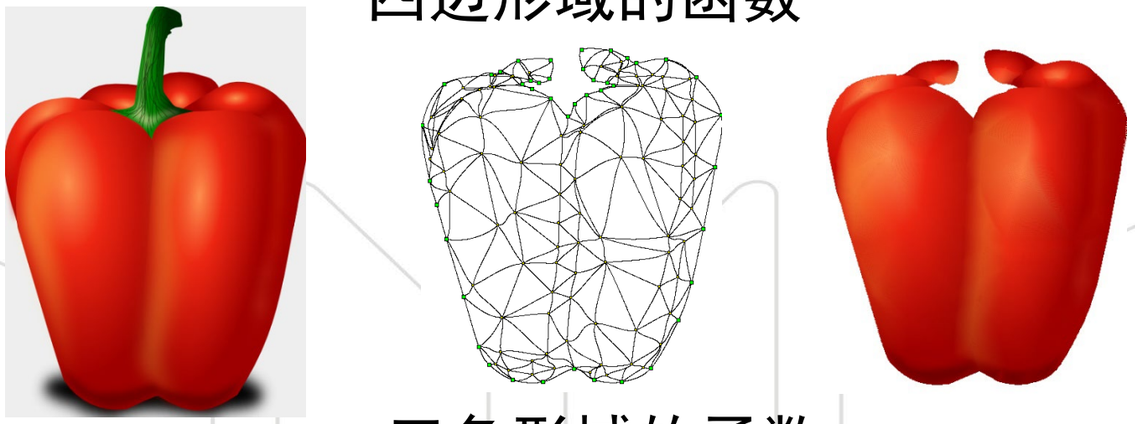

Rank 150


原图

- 如何理解图像？
  - 图像是一个函数
    - 可使用网格对其近似

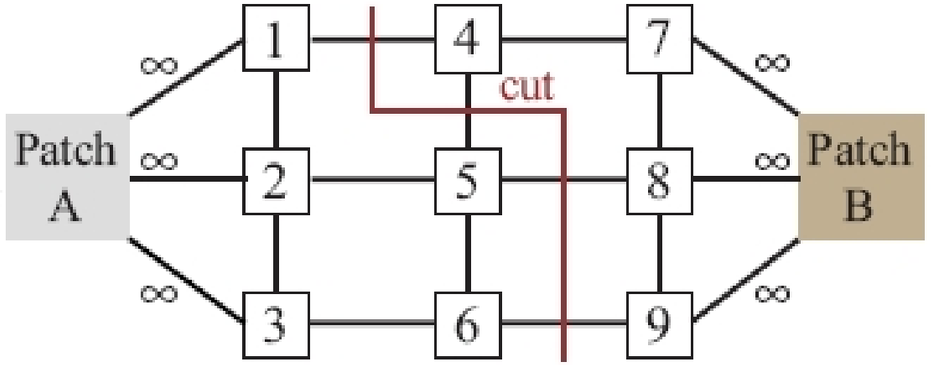


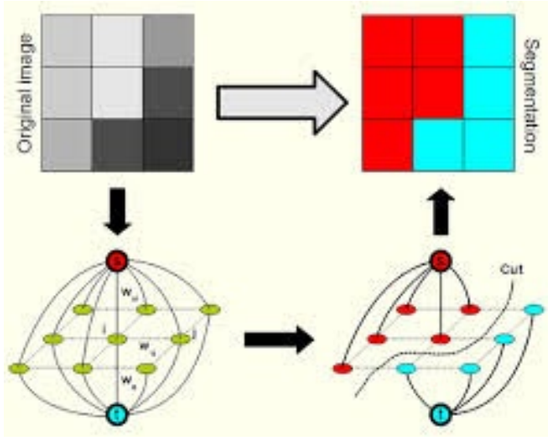
四边形域的函数



三角形域的函数

- 如何理解图像？
  - 图像是一个图
    - 可使用图算法（如最大流最小割）对其进行分割





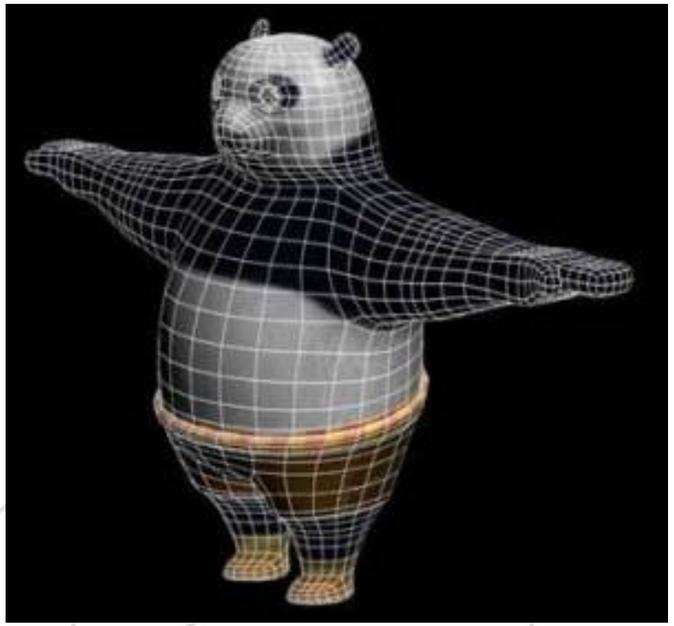




- 基本范式

基于采样的图形 VS 基于几何的图形

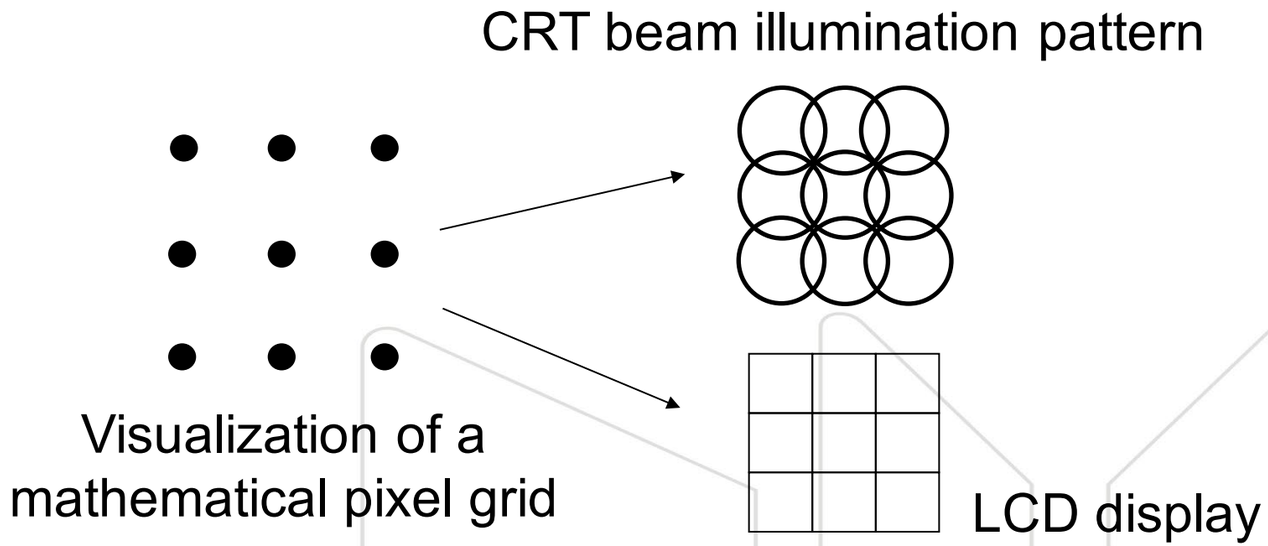


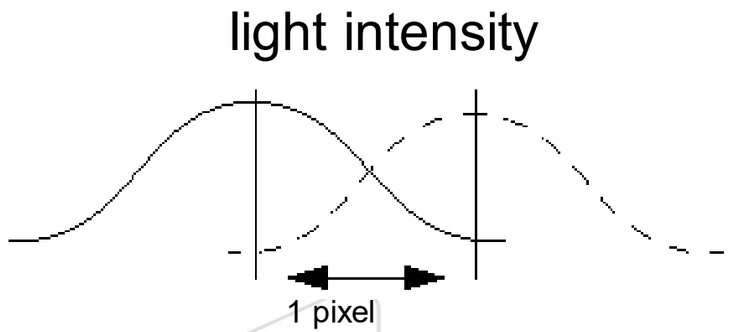


- 基于采样的图形vs基于几何的图形
  - 使用离散采样点描述视觉信息
    - 像素可由数码图像产生，并使用基于采样的程序绘制
    - 数码图像不仅是照片，也包括描述物理世界的标量场
      - 如，气温场
      - 常用于可视化
    - 基于采样的程序：Adobe Photoshop, GIMP



- 基于采样的图形vs基于几何的图形
  - 像素由采样点位置及其取值构成
    - 光照强度、颜色、透明度、及其他控制信息，等
  - 图像是对连续信号的离散采样
    - 而非一个个小圆点或方块

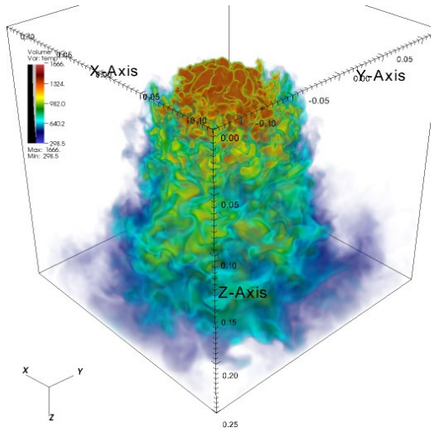




Can’t visually resolve adjacent pixels on CRT

计算机图形系统

- 基于采样的图形vs基于几何的图形
  - 由绘制程序直接产生采样点
  - 由其他设备产生的连续模拟信号中采样得到采样点
    - 以固定间隔测量的光照强度/颜色
    - 平板或滚筒扫描仪
    - 数码相机、数码摄像机
  - 由数值输入中采样
    - 如，模拟数据集





  - 图像可定义为像素（采样点）数组，并进行相应操作
    - 图片编辑：由用户手动修改图像，如切割、粘贴、或使用画笔工具在图像上进行绘制，等
    - 图像处理：使用算法自动在图像上完成操作，如模糊、锐化、边缘检测、白平衡，及其他计算机视觉操作，等。

计算机图形系统

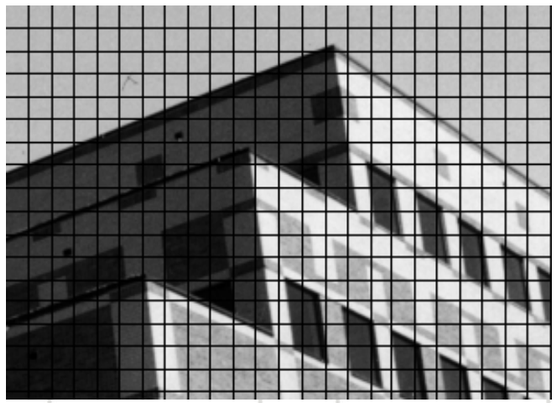
- 基于采样的图形vs基于几何的图形
  - 图像采样举例
  - 对右图进行的建筑物进行采样



  - 在网格中的每个格点上采样一个颜色值
    - 使用该颜色值给整个格子上色
    - 可能产生“块状”的采样图像

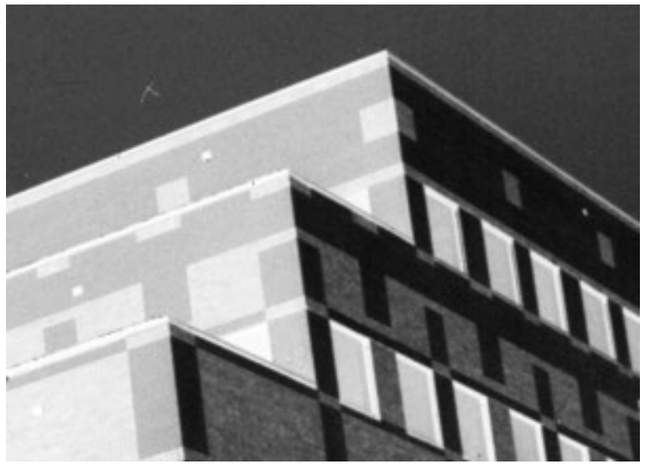
0 = white, 5 = gray, 10 = black

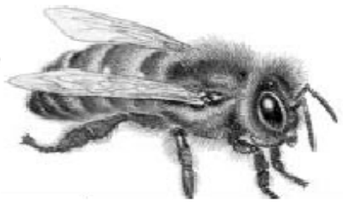






- 基于采样的图形vs基于几何的图形
  - 基于采样的优点：易于操作
    - 图像定义在规则网格上，可对每个格点的颜色或位置进行修改
      - 如，对此前的图片取反色，只需将10映射至白色，0映射至黑色即可
    - 合并两张图像时，只需将一张图像中像素值拷贝至另一张图像的相应位置即可（替换或结合）

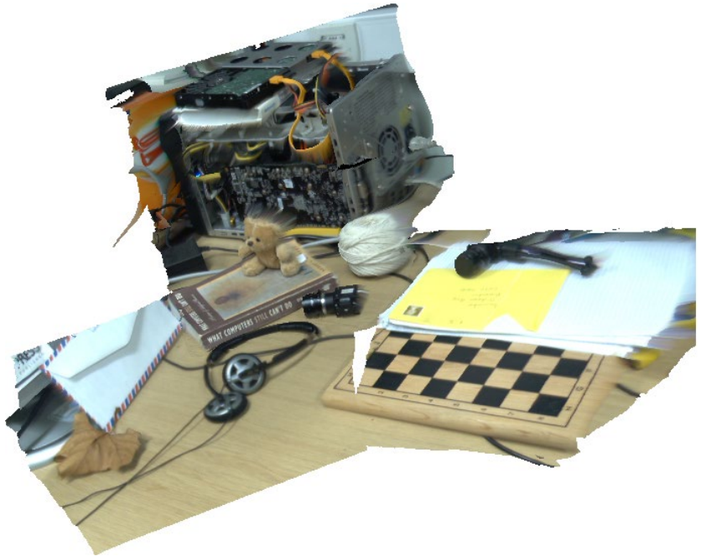


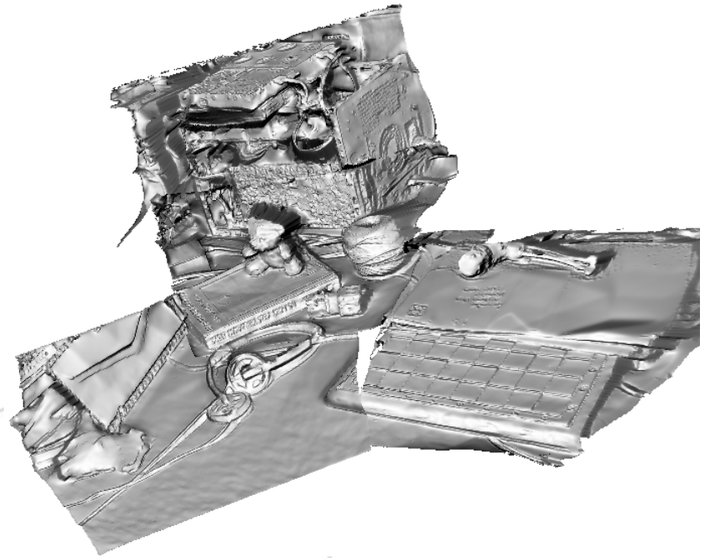




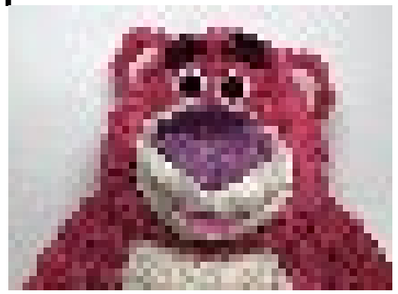


- 基于采样的图形vs基于几何的图形
  - 基于采样的缺点：WYSIAYG
    - What you see is all you get.
    - 缺乏额外信息
      - 如，空间位置、深度等
    - 无法更换观察视角
    - 只能对像素进行操作
      - 更改颜色、增强对比度等
    - 尽管当前也出现了很多基于图像产生伪3D效果（甚至三维重建）的方法
      - 在一定范围内更改视角
      - 使用插值、变形、及其他方法融合多张图像



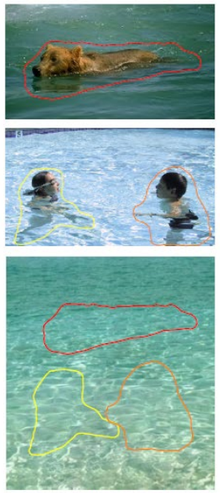


- 基于采样的图形vs基于几何的图形
  - 某些操作需要特定算法完成
    - 超分辨率（super-resolution）：从低分辨率图像产生高分辨率图像





    - 图像融合：将一张图像中的物体无缝嵌入到另一张图像中



sources/destinations

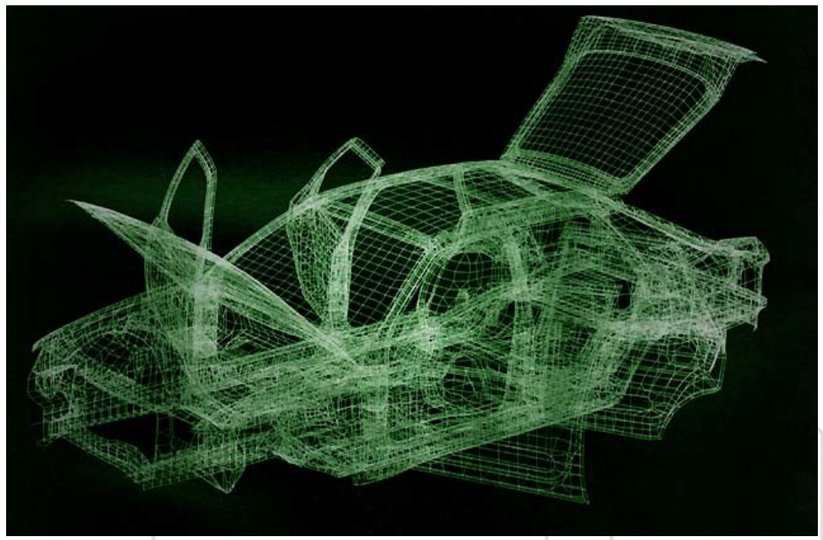


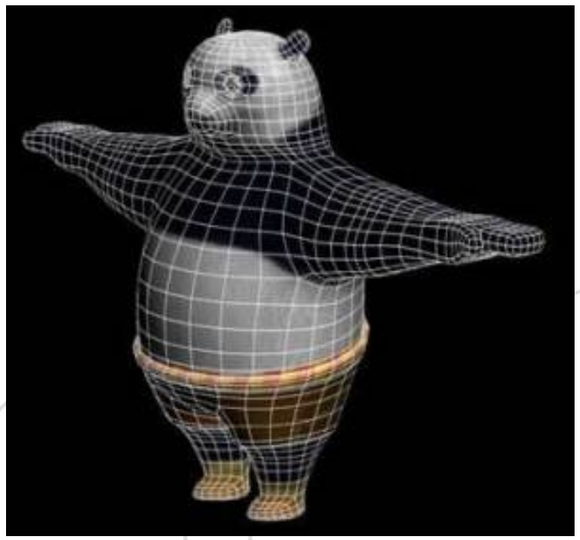
cloning



seamless cloning

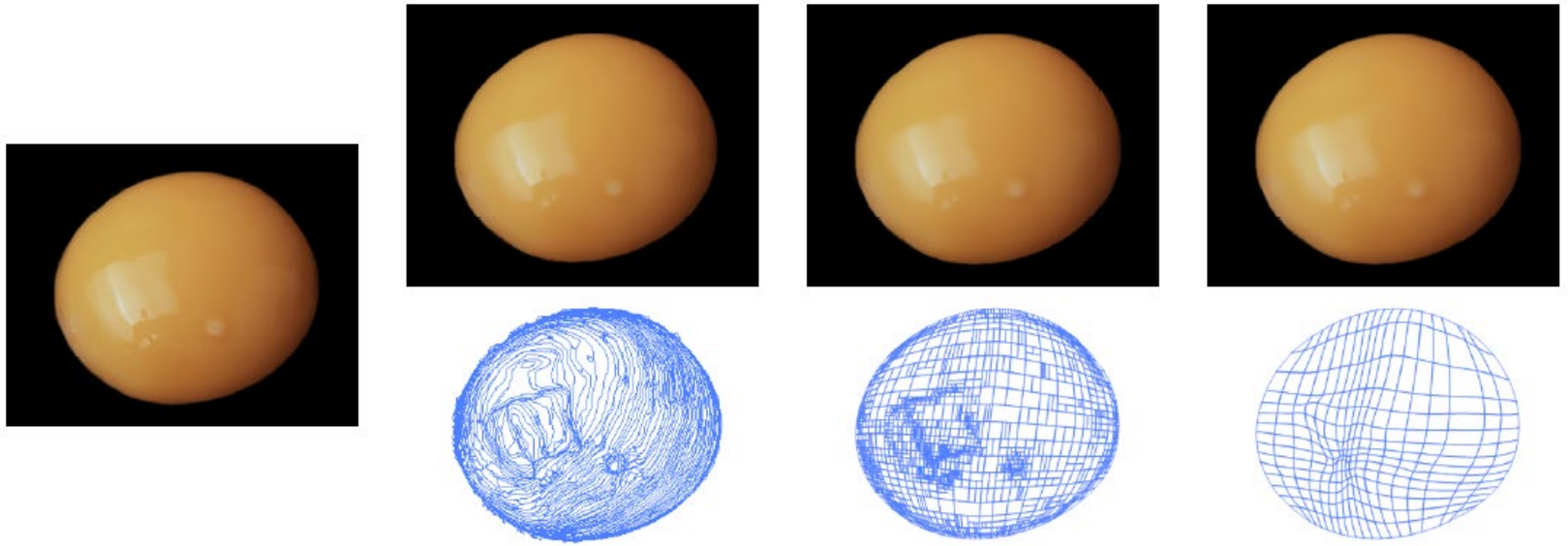
- 基于采样的图形vs基于几何的图形
  - Scalable vector graphics 或 object-oriented graphics
    - 创建几何对象（及其附属外观属性），并通过计算机图形学或可视化方法采样后进行渲染及显示
    - 常通过模拟物理世界完成
    - 2D工具：Adobe Illustrator, Adobe Freehand, Corel CorelDRAW
    - 3D工具：Autodesk AutoCAD, Autodesk Maya, POV-Ray





- 基于采样的图形vs基于几何的图形
  - 基于几何的图形应用
    - 存储几何体的数学表现形式（模型）或几何元素（点、线、多边形、多边形网格，等）及其对应的属性（颜色、材料属性，等）
    - 几何元素即之前所提到的“primitives（图元）”
    - 图像尽管可能由几何体采样得到，但其往往并非以模型的形式存储
    - 在基于几何的应用程序中，用户通常不直接对每个像素进行操作，而是对几何元素进行操作，由程序自动对其进行重采样并重新绘制图像

- 基于采样的图形vs基于几何的图形
  - 同时，许多方法正使用基于几何以及基于采样的方法融合的方式进行渲染，以提高渲染速度甚至最终生成图像的质量
    - 在图像上绘制几何体（3D人物等）
    - 在3D场景中使用纹理贴图等
  - 从基于采样到基于几何的图形



(a) input raster image (b) Adobe Live Trace result (c) subdivision mesh (d) optimized gradient mesh

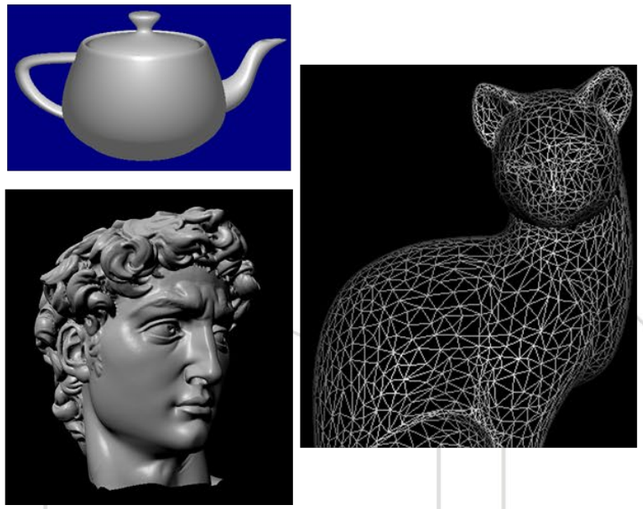
课程概要

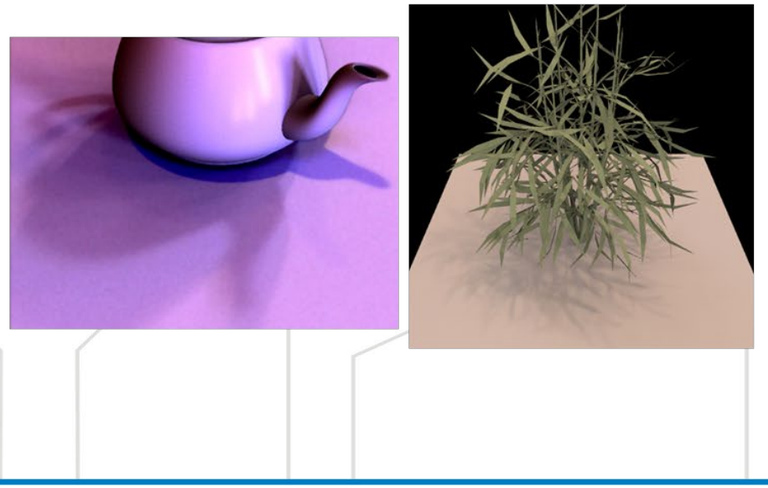
- 计算机图形系统
- 物理成像系统
- 图形渲染管线

Modeling
(Creating 3D Geometry)

→

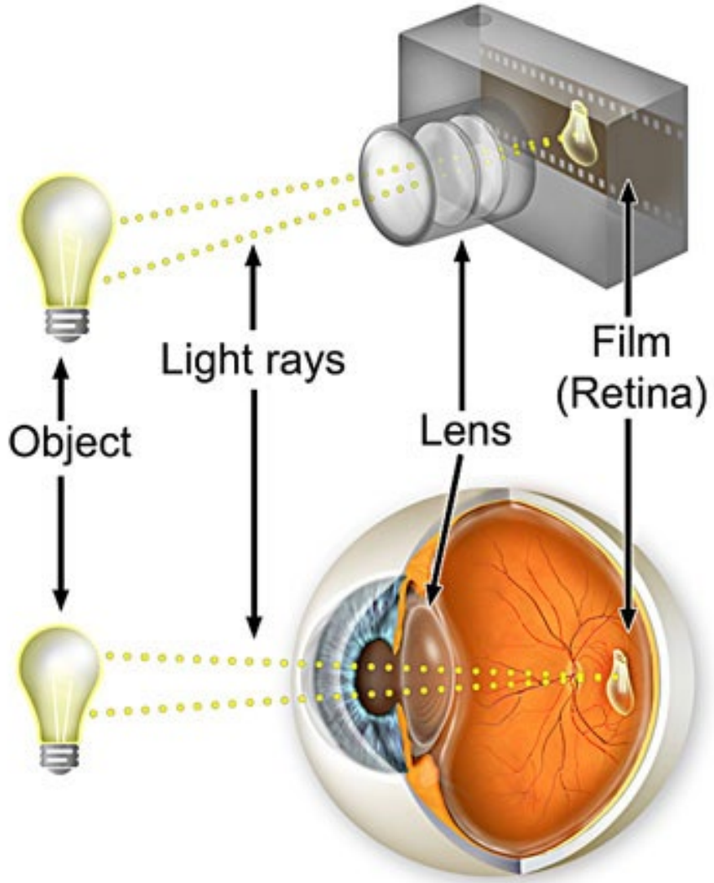
Rendering
(Creating, shading images from geometry, lighting, materials)

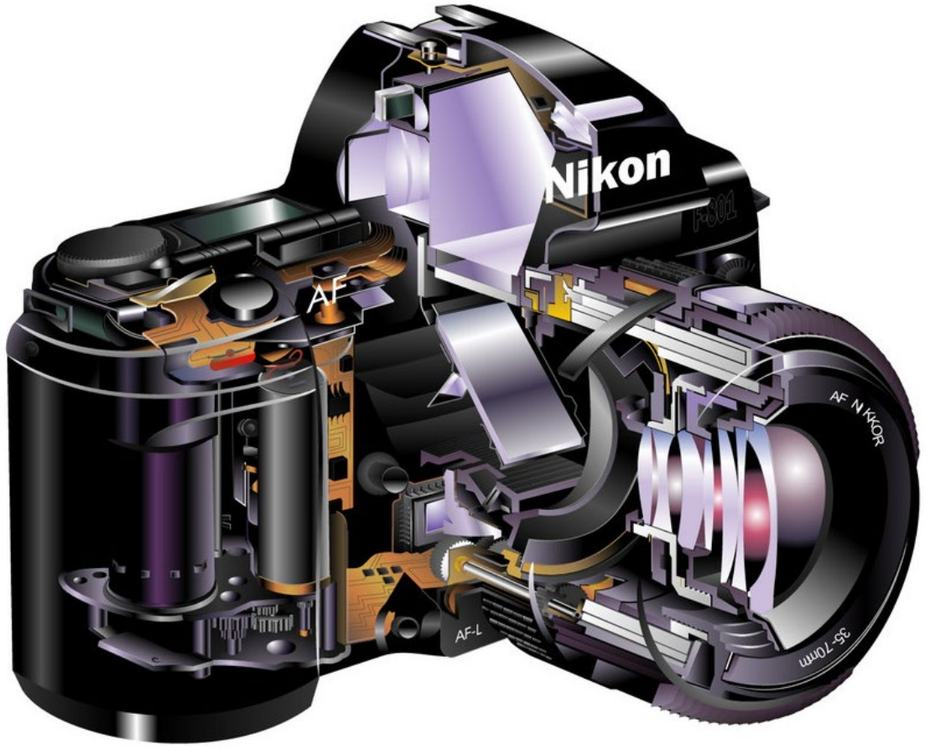




物理成像系统

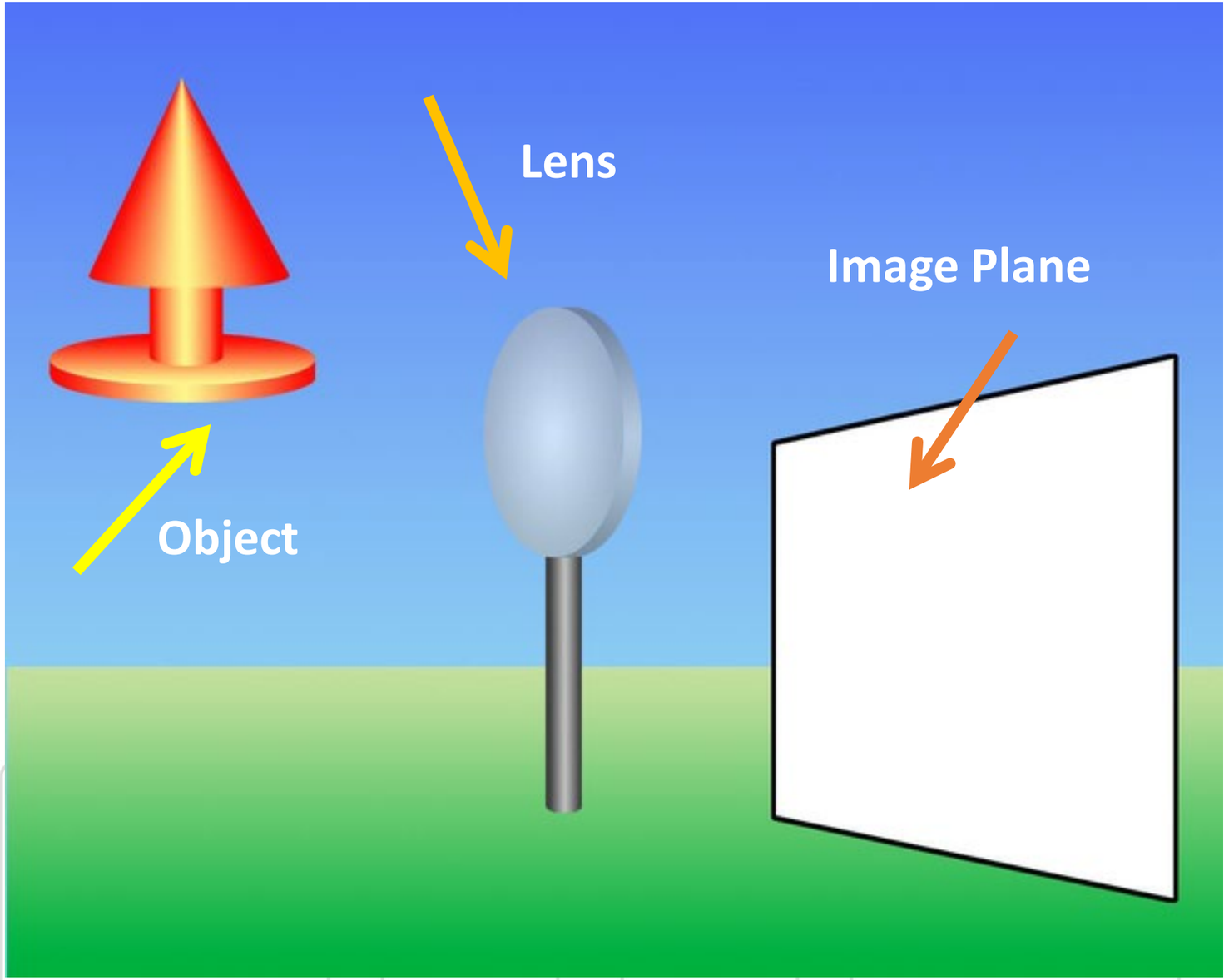
- 眼睛（生物学）
- 相机：胶片（物理+化学），数码（物理+信息学）





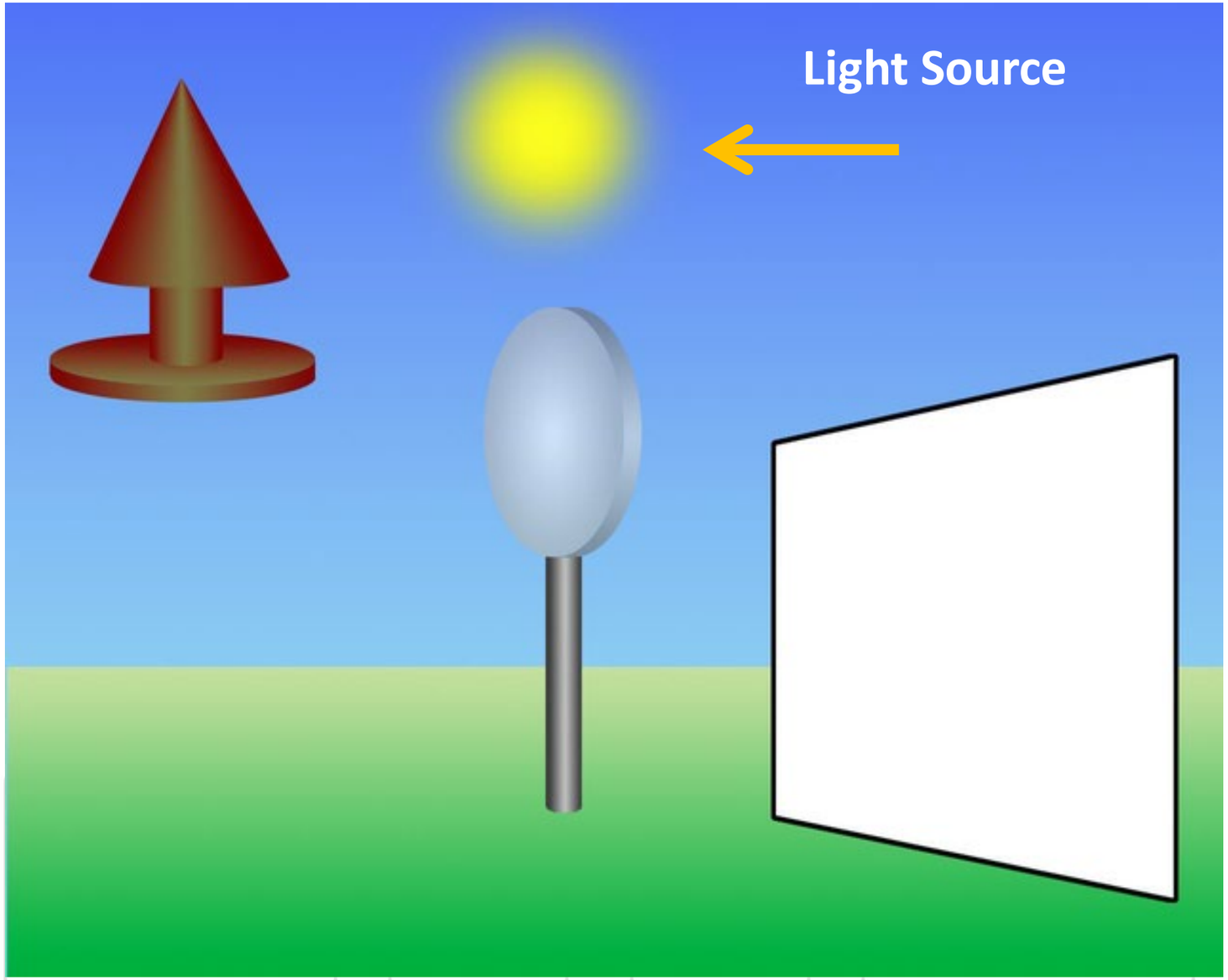
物理成像系统

- 成像原理



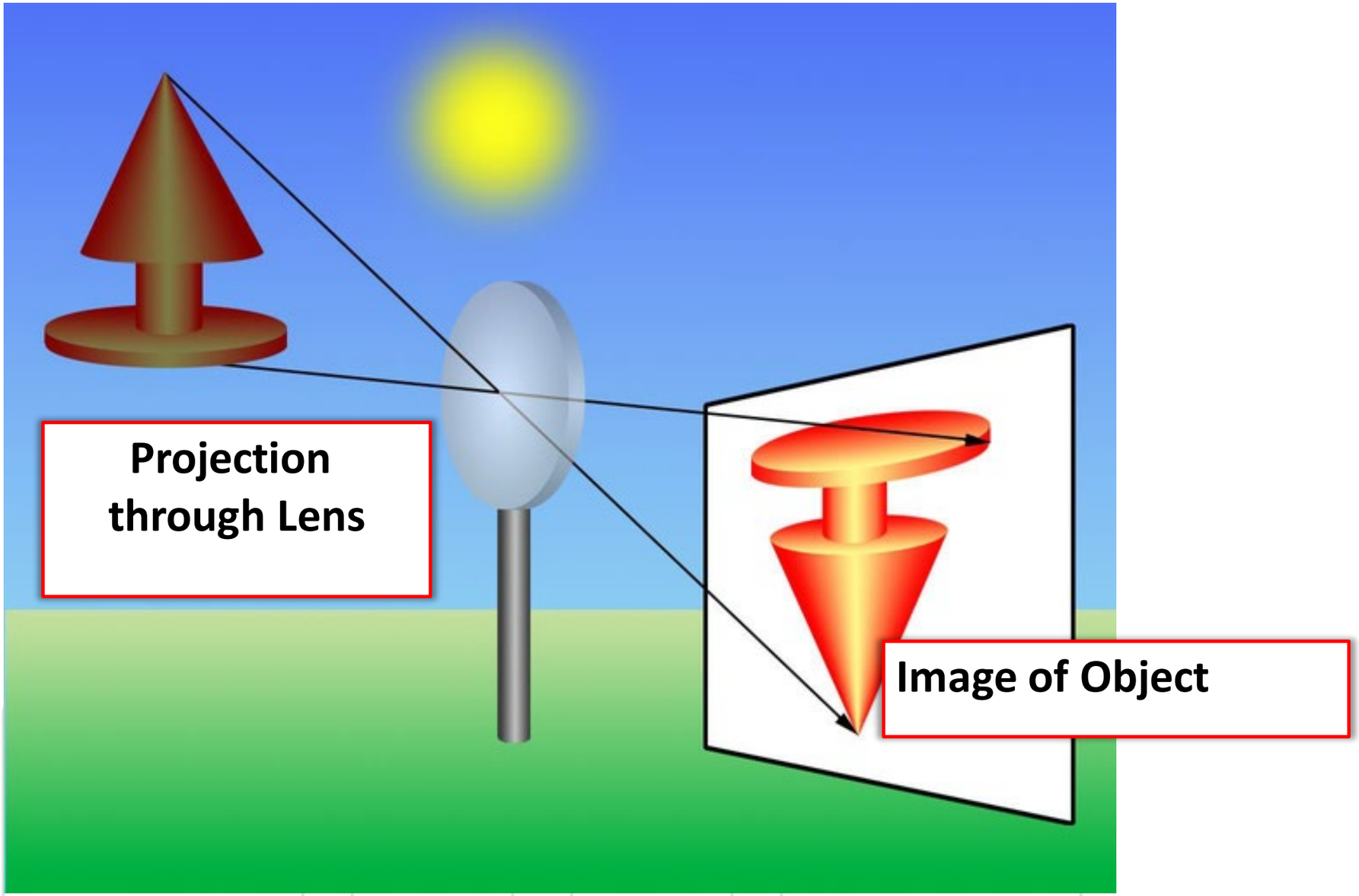
物理成像系统

- 成像原理



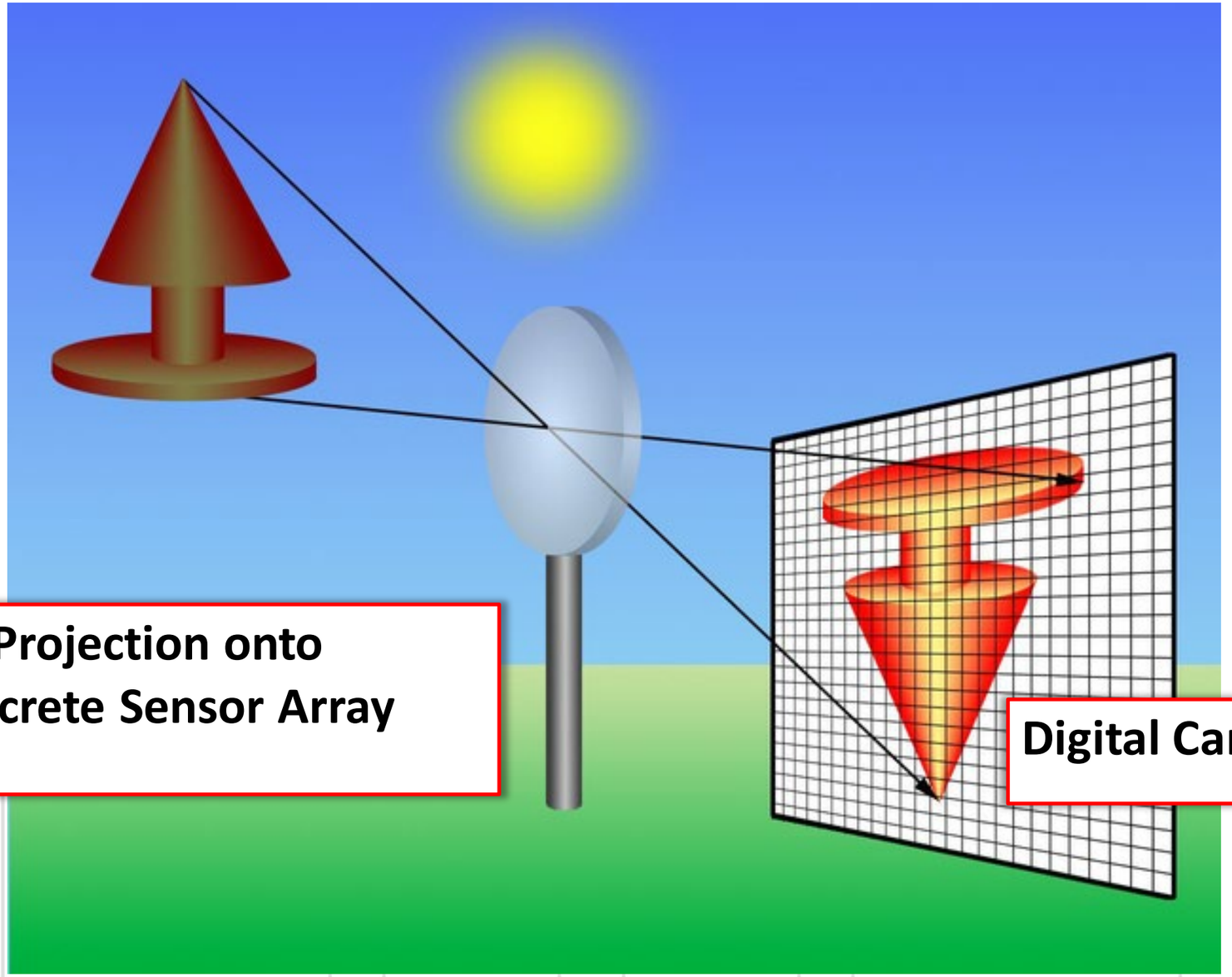
物理成像系统

- 成像原理



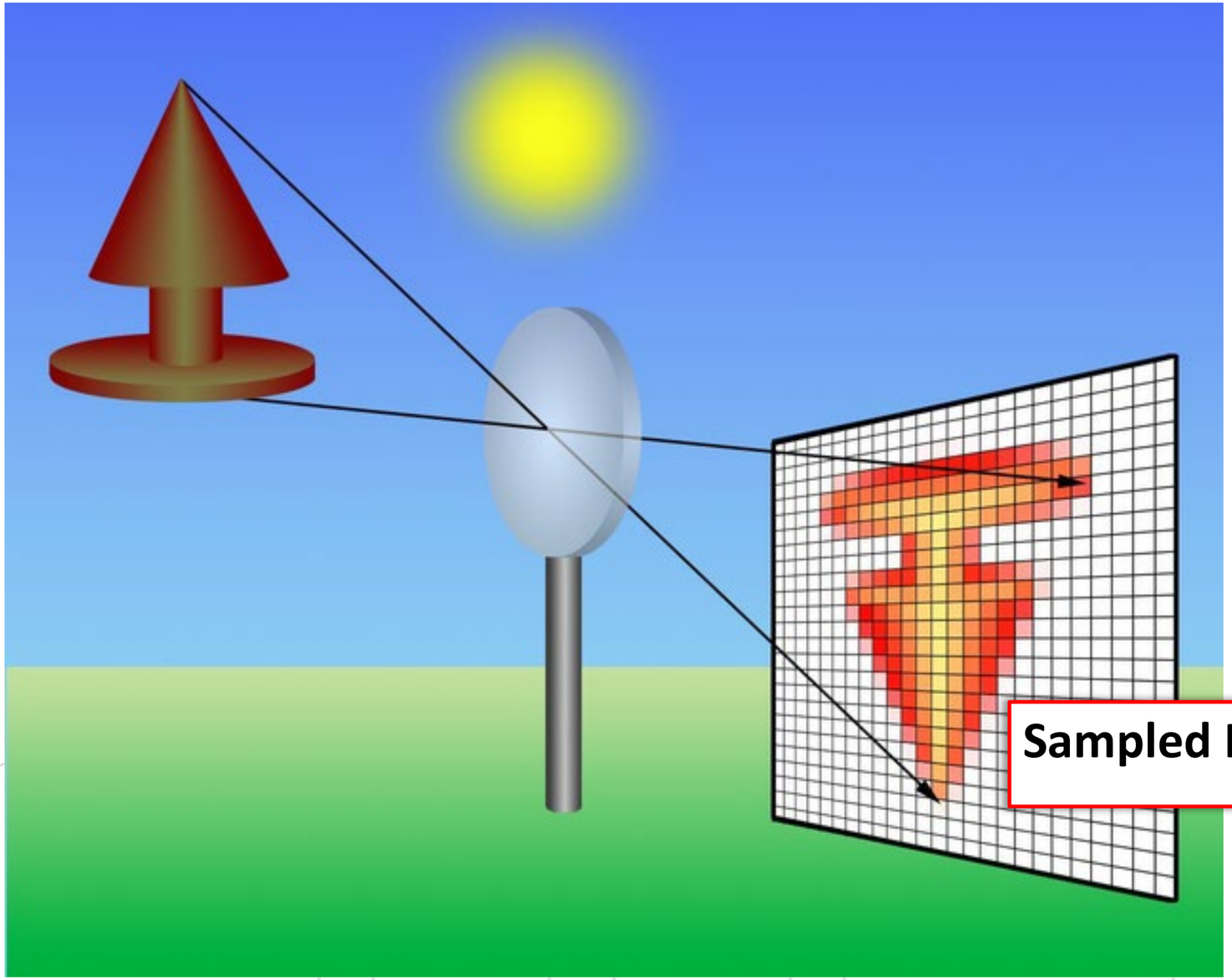
物理成像系统

- 成像原理



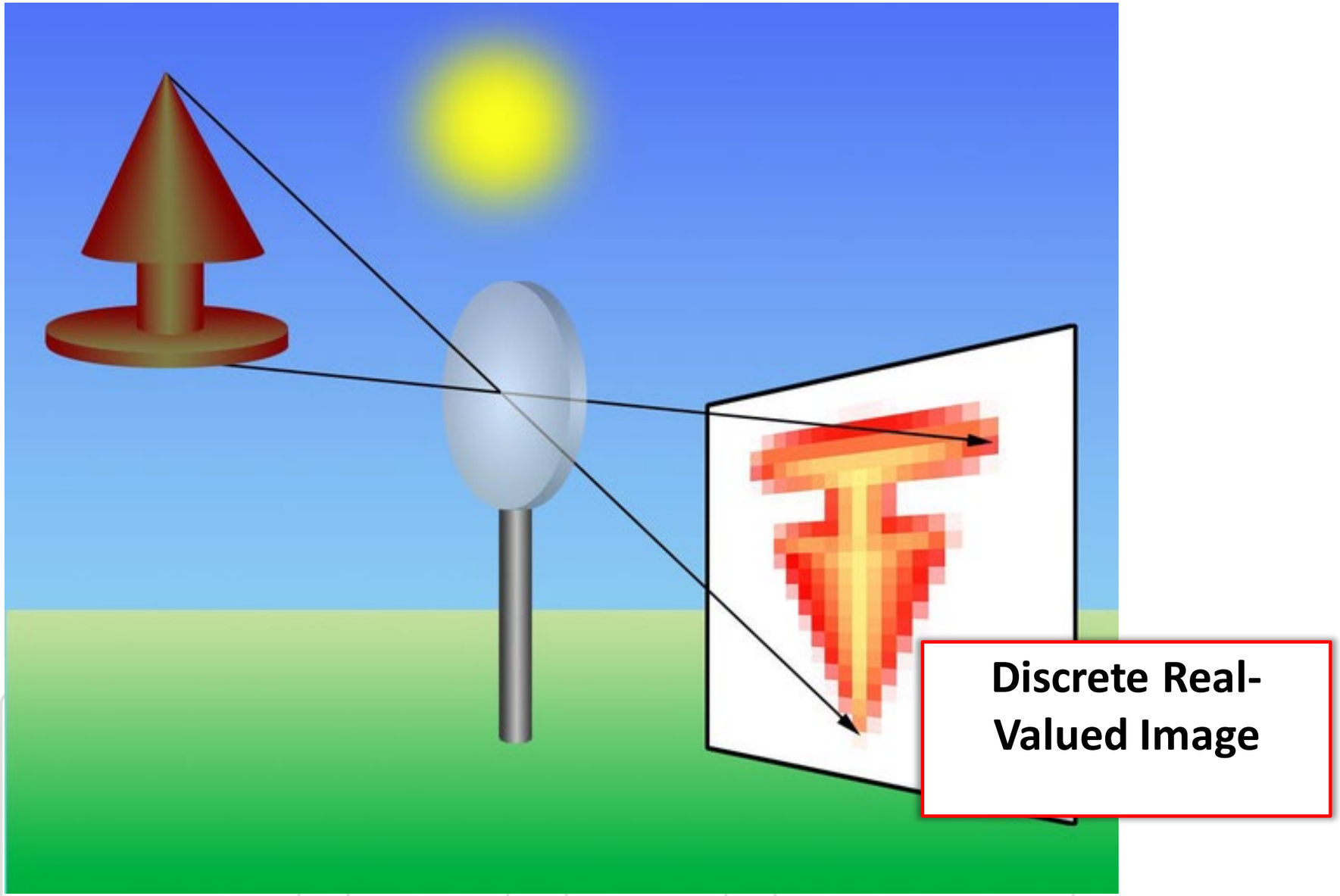
物理成像系统

- 成像原理



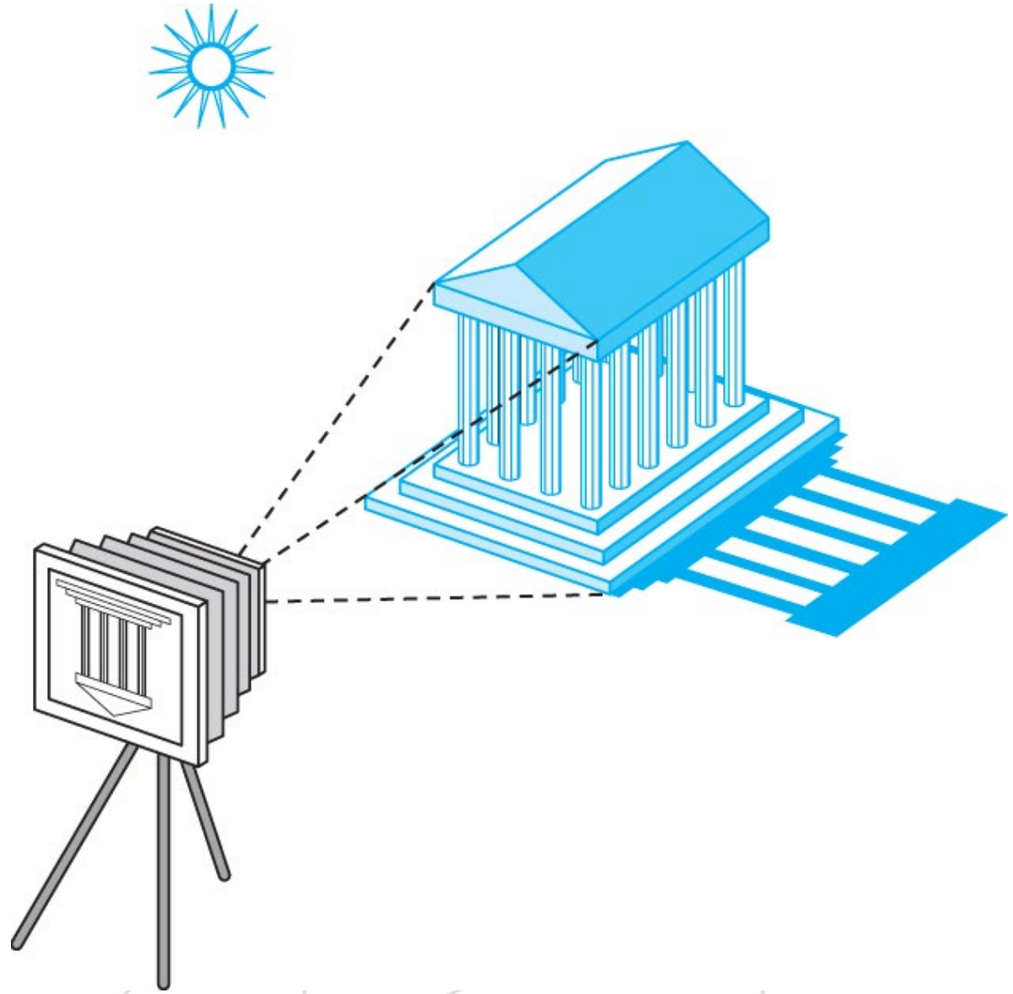
物理成像系统

- 成像原理



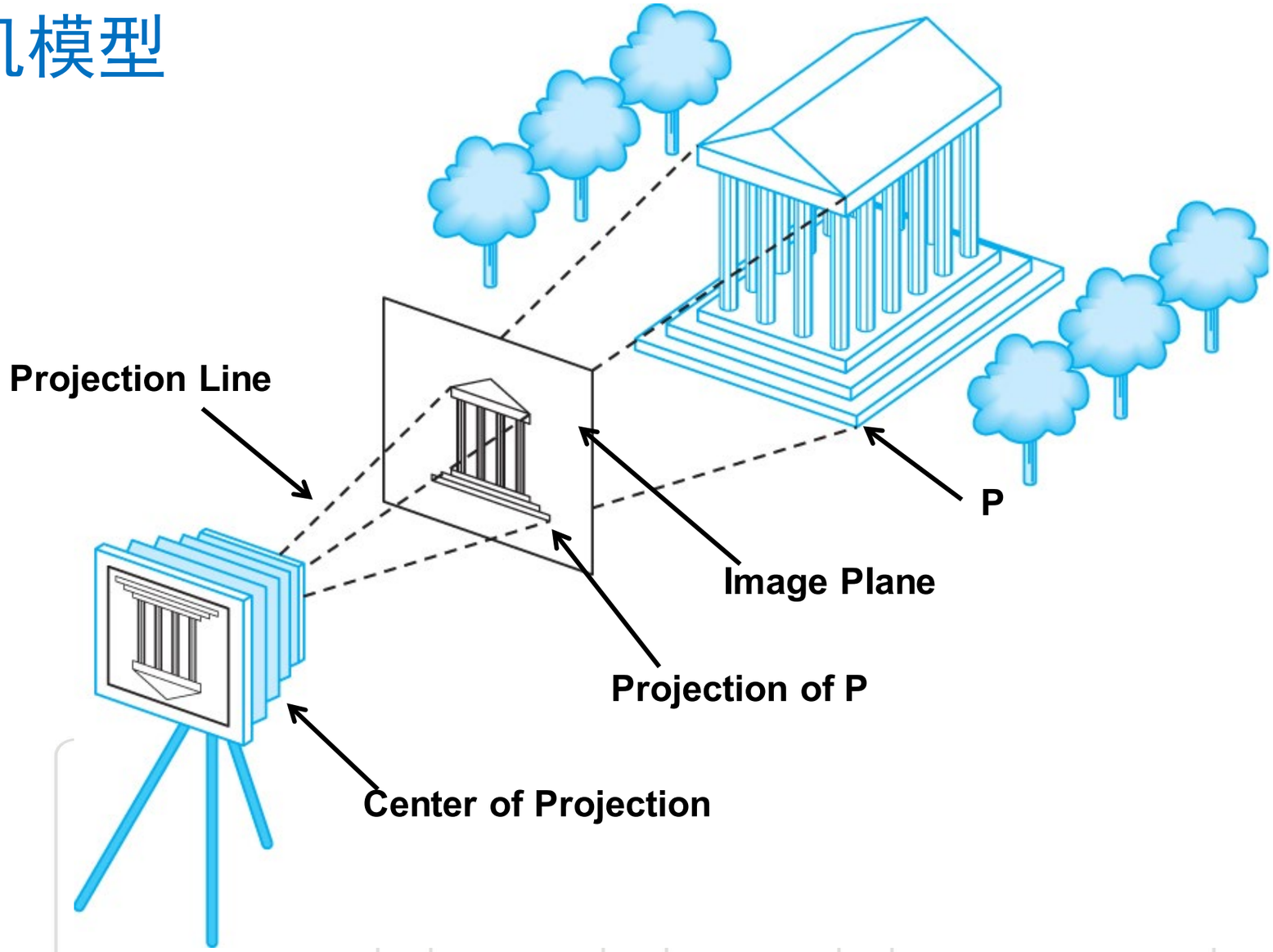
物理成像系统

- 生成图像的基本元素
  - 物体
  - 观察者
  - 光源
  - 及其属性
    - 例如，光源与物体材质的属性将决定光照在物体上所产生的视觉效果



物理成像系统

- 模拟相机模型



物理成像系统

- 模拟相机模型优点
  - 物体、观察者、光源完全独立
  - 2D图形可视为3D图形的一个特例
  - 易于使用API实现
    - 设置光源、相机、材质属性
    - 其余部分交由API完成（基础图形学）
  - OpenGL、Direct3D等API即基于此模型并提供快速硬件实现

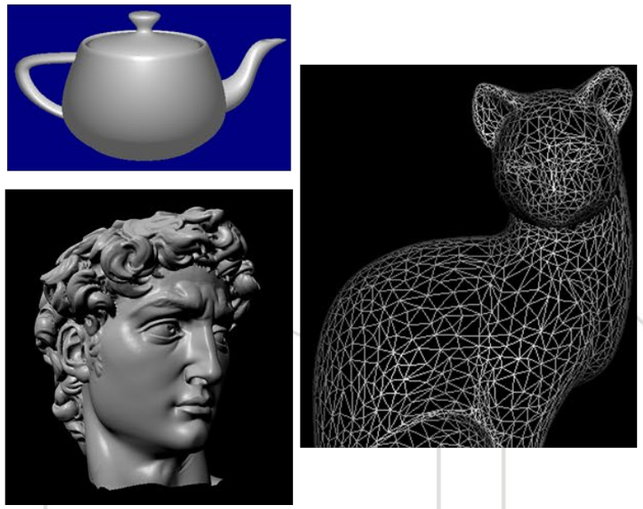
课程概要

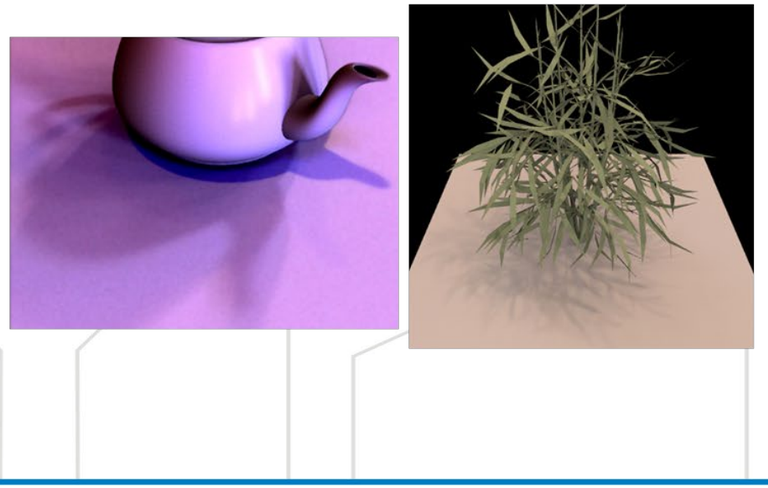
- 计算机图形系统
- 物理成像系统
- 图形渲染管线

Modeling
(Creating 3D Geometry)

→

Rendering
(Creating, shading images from geometry, lighting, materials)



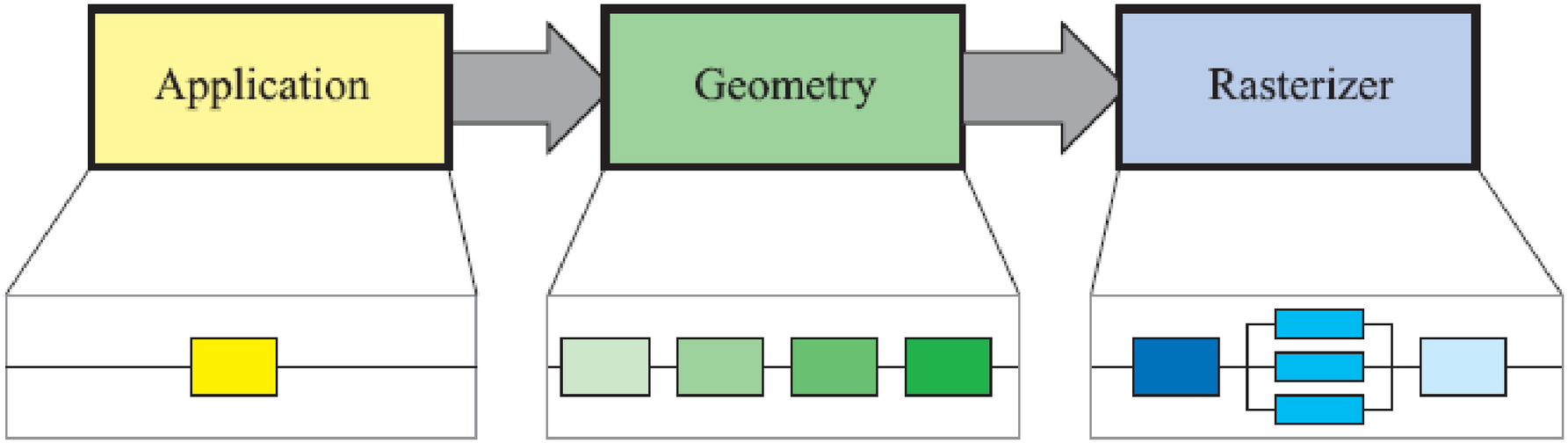


- Demo – World War Z



图形渲染管线

- 渲染定义
  - “Rendering is a process that takes as its input a set of objects and produces as its output an array of pixels.”
    - Fundamentals of Computer Graphics (3rd Edition)
  - 这一渲染流程即为图形渲染管线
    - 将3D场景转换为2D光栅化表现形式（图像）的一系列步骤



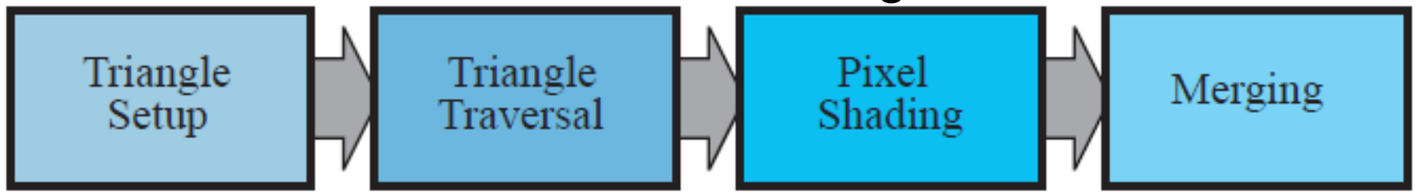
图形渲染管线

渲染管线步骤

- Application（应用）：CPU（及GPU）完成
  - 碰撞检测，物理模拟，全局加速算法，动画
- Geometry（几何）：GPU完成
  - 完成几何变换；计算需要绘制的内容，绘制的方式，以及绘制的位置

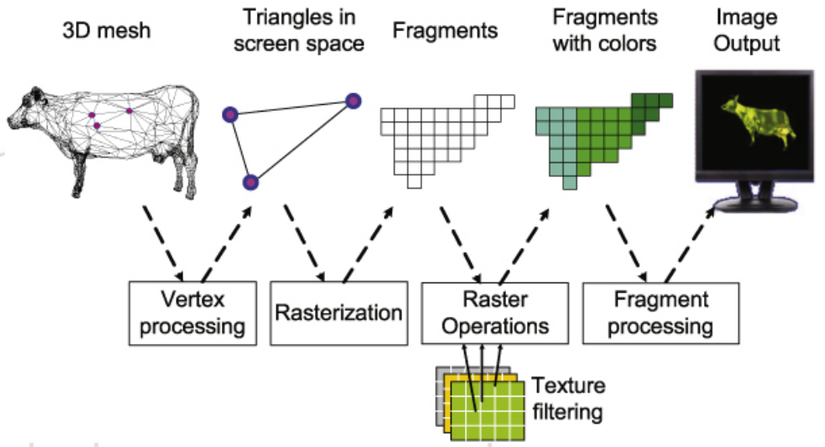


- Rasterizer（光栅）：GPU完成
  - 将屏幕空间中的二维顶点转换为像素
    - 考虑顶点的z-value（深度）及相关shading属性



图形渲染管线

- 渲染管线
  - 抽象模型
    - 从几何模型到数字图像的一系列操作
    - 图形硬件的工作流程
  - 可使用API对图形硬件进行编程
    - OpenGL
    - Direct 3D
    - 已有许多现成的实现



- NVIDIA Samples
  - https://developer.nvidia.com/gameworks-samples-overview

NVIDIA GameWorks™ OpenGL Samples

Get the documentation or download the NVIDIA GameWorks™ OpenGL samples here:

Download >

New Browse or Clone Source from GitHub:

GitHub >

New samples in 2.0:

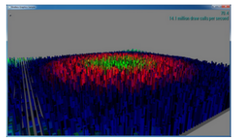
- Blended Antialiasing Sample
- Cascaded Shadow Mapping Sample
- Conservative Rasterization Sample
- Normal-Blended Decal Sample
- Weighted, Blended, Order-independent Transparency Sample

Bindless Graphics Sample

- Category: **Performance**

This sample demonstrates the large performance increase in OpenGL that is made possible by 'Bindless Graphics'. These extensions allow applications to draw large numbers of objects with only a few setup calls, rather than a few calls per object, thus reducing the driver overhead necessary to render highly populated scenery.

Docs >

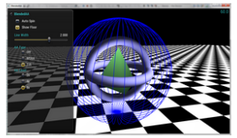


★NEW:Blended AA

- Category: **Performance, Visuals**

This sample implements a two-pass additive blending anti-aliasing technique using Target-Independent Rasterization (TIR), which should give comparable results to MSAA with a reduced memory footprint.

Docs >

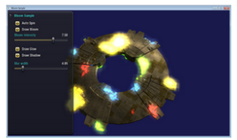


Bloom Sample

- Category: **Visuals**

This sample demonstrates creating a glow effect by post-processing the main scene. It heavily leverages FBO render targets across multiple steps/passes with custom effects processing shaders. It also integrates shadow mapping to demonstrate self-illumination cutting through the shadow effects.

Docs >



NVIDIA GameWorks™ DirectX Samples

Get the documentation or download the NVIDIA GameWorks™ Direct3D samples here.

Download >

New Browse or Clone Source from GitHub:

GitHub >

New samples in 1.2:

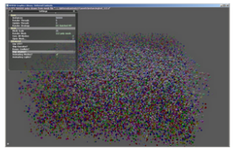
- Antialiased Deferred Rendering
- Motion Blur D3D Advanced Sample

D3D Deferred Contexts Sample

- Category: **Performance**

This sample shows how to use D3D11 Deferred Rendering contexts to lower the CPU overhead and improve performance when rendering large numbers of objects per frame, in situations where instancing is not feasible.

Docs >

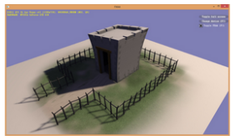


FXAA 3.11 Sample

- Category: **Performance,Visuals**

This sample presents a high performance and high quality screen-space software approximation to anti-aliasing called FXAA.

Docs >

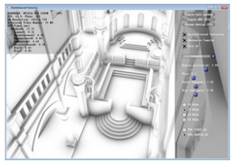


Deinterleaved Texturing Sample

- Category: **Performance,Visuals**

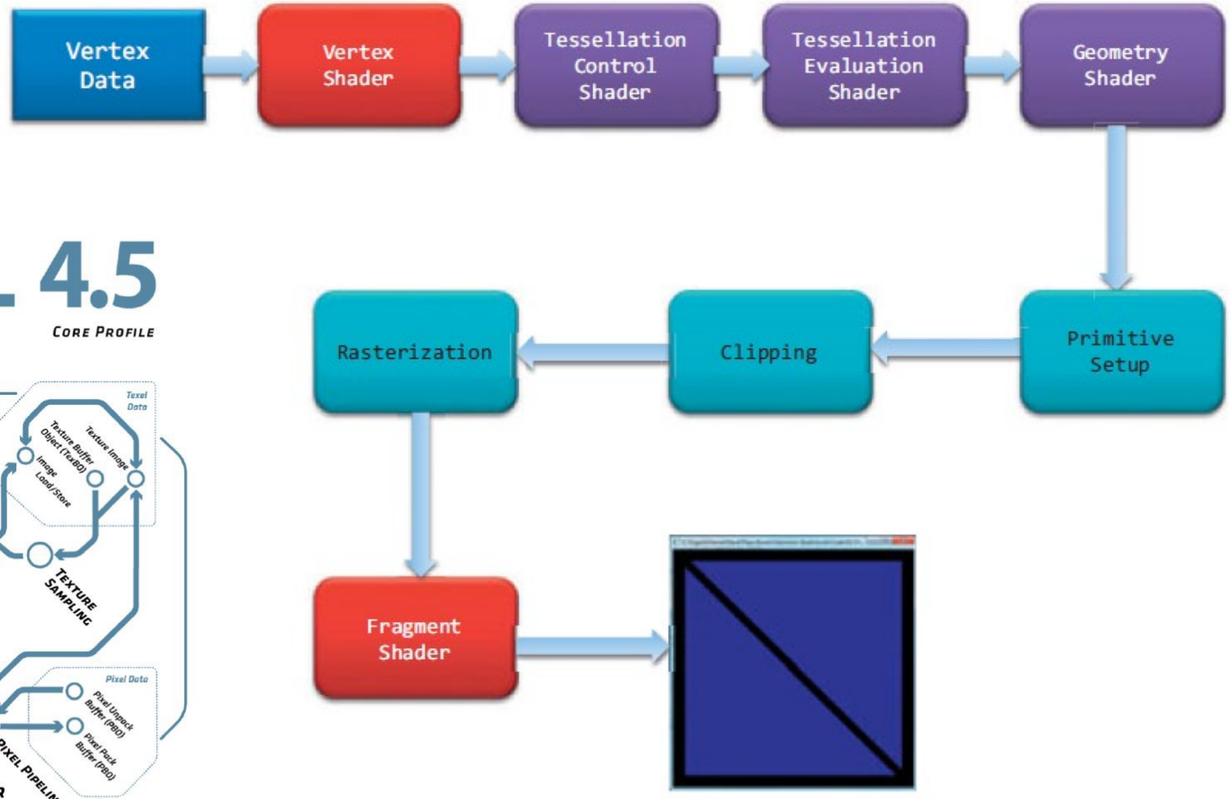
This sample demonstrates how a large, sparse and jittered post-processing filter (here a SSAO pass with a 4x4 random texture) can be made more cache-efficient by using a Deinterleaved Texturing approach.

Docs >



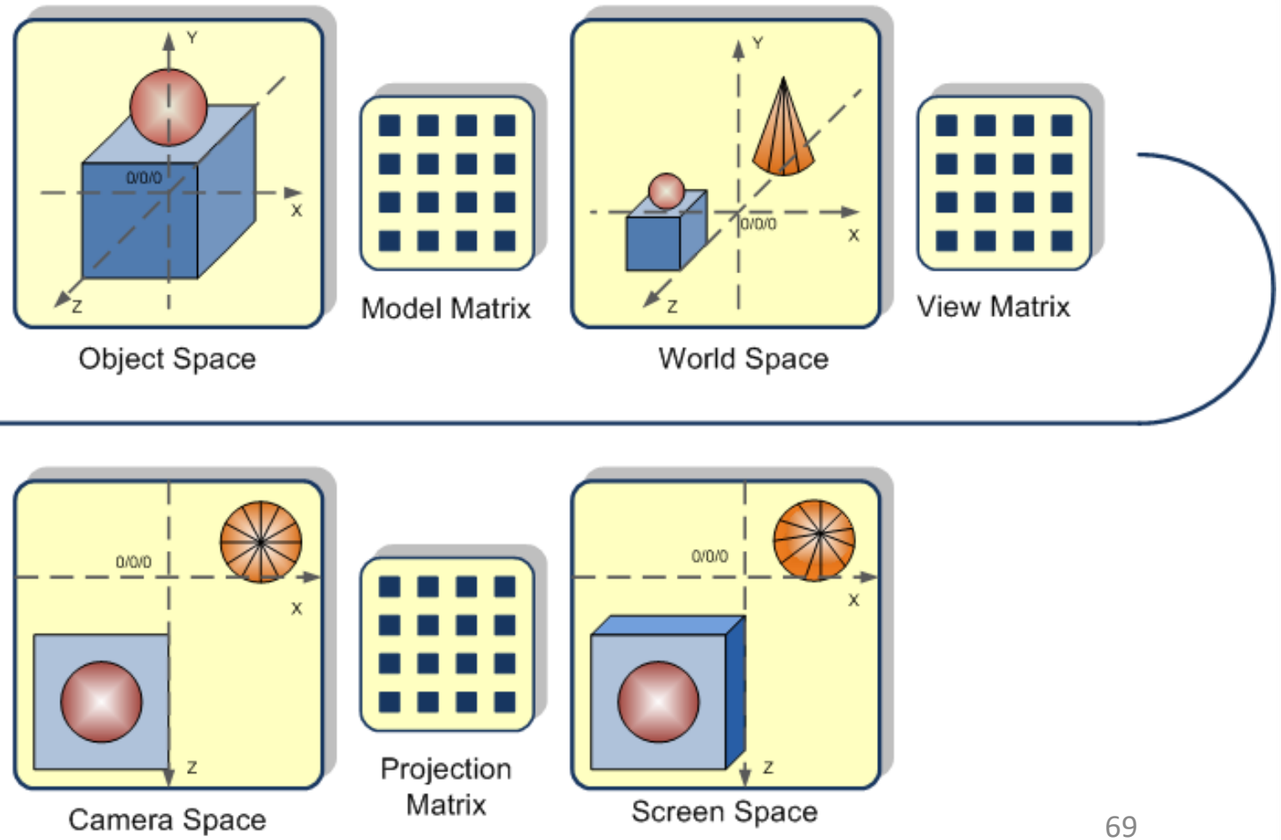
图形渲染管线

- 渲染管线与着色器（shader）
  - 用户可编程对着色器的行为进行更改
    - 如，如何将点绘制为三维效果的小球？



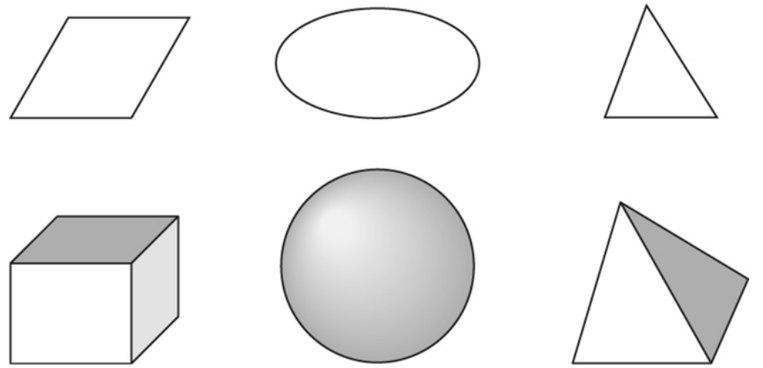
图形渲染管线

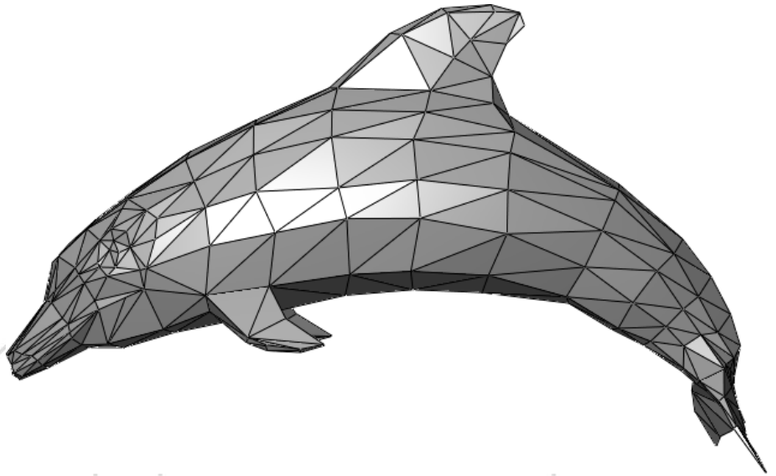
- 顶点处理
  - 1. object space中的物体
  - 2. 通过model matrix变换至world space
  - 3. 通过view matrix变换至camera (eye) space
  - 4. 通过projection matrix变换至screen space



图形渲染管线

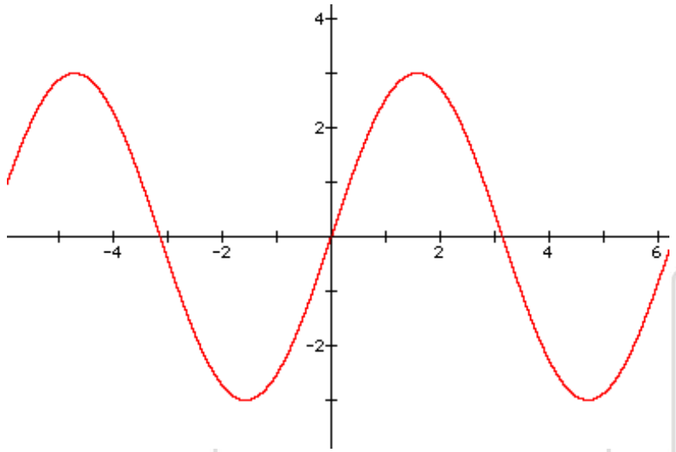
- 几何图元
  - 简单几何体
    - 球体、椎体、圆柱体、圆环，等
    - POV-Ray
  - 简单图元
    - 三角形及三角形网格
    - OpenGL
  - 复杂图元及其参数
    - Spline，NURBS，等
    - 隐函数（implicit functions）
      - 如，$x^2 + y^2 - 1 = 0$

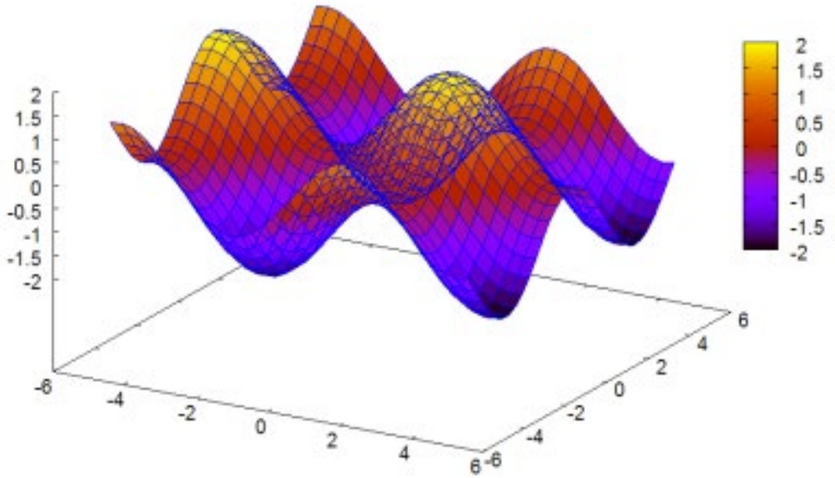




图形渲染管线

- 几何图元（复杂图元及其参数）
  - 显函数（explicit functions)
    - 曲线：如，$y = \sin(x)$
    - 曲面：如，$y = \sin(x) + \cos(y)$
    - 难以表达三维中的任意形状



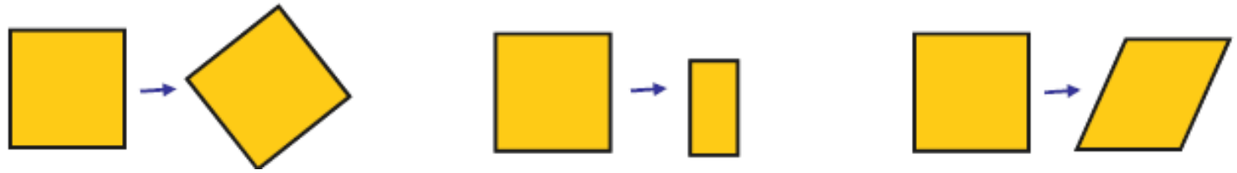


- 几何图元（复杂图元及其参数）
  - 参数函数（parametric functions)
    - 曲线：x, y, z 均为参数 t 的函数
    - 曲面：x, y, z 均为参数 s, t 的函数
    - 参数名可以为任意字符
    - 以下两个参数函数分别是什么图形？
    - $C(t) \coloneqq \begin{pmatrix} \cos(t) \\ \sin(t) \\ t \end{pmatrix}$
    - $S(\phi, \theta) \coloneqq \begin{pmatrix} \cos\phi\cos\theta \\ \sin\phi\cos\theta \\ \sin\theta \end{pmatrix}$

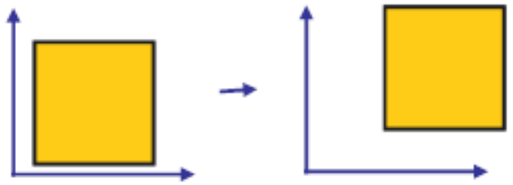
图形渲染管线

Model/View变换

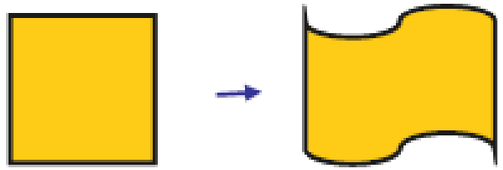
- 变换类型
  - 旋转（rotation）、缩放（scaling）、剪切（shearing）

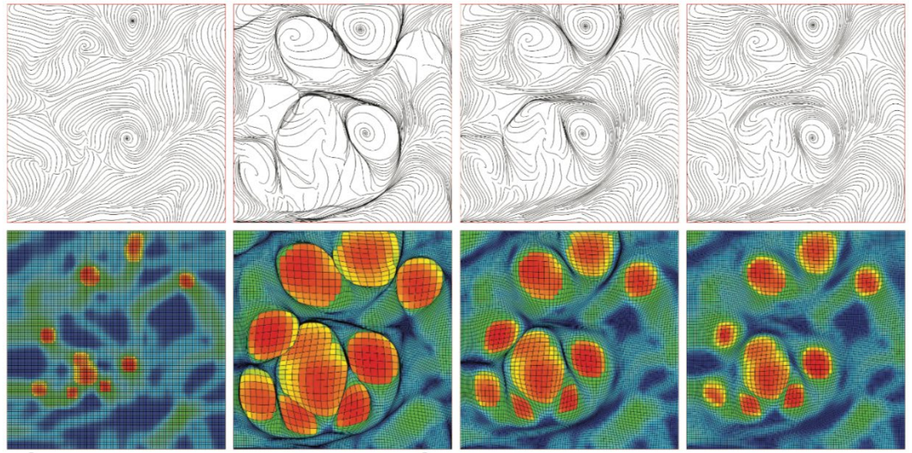


  - 平移（translation）



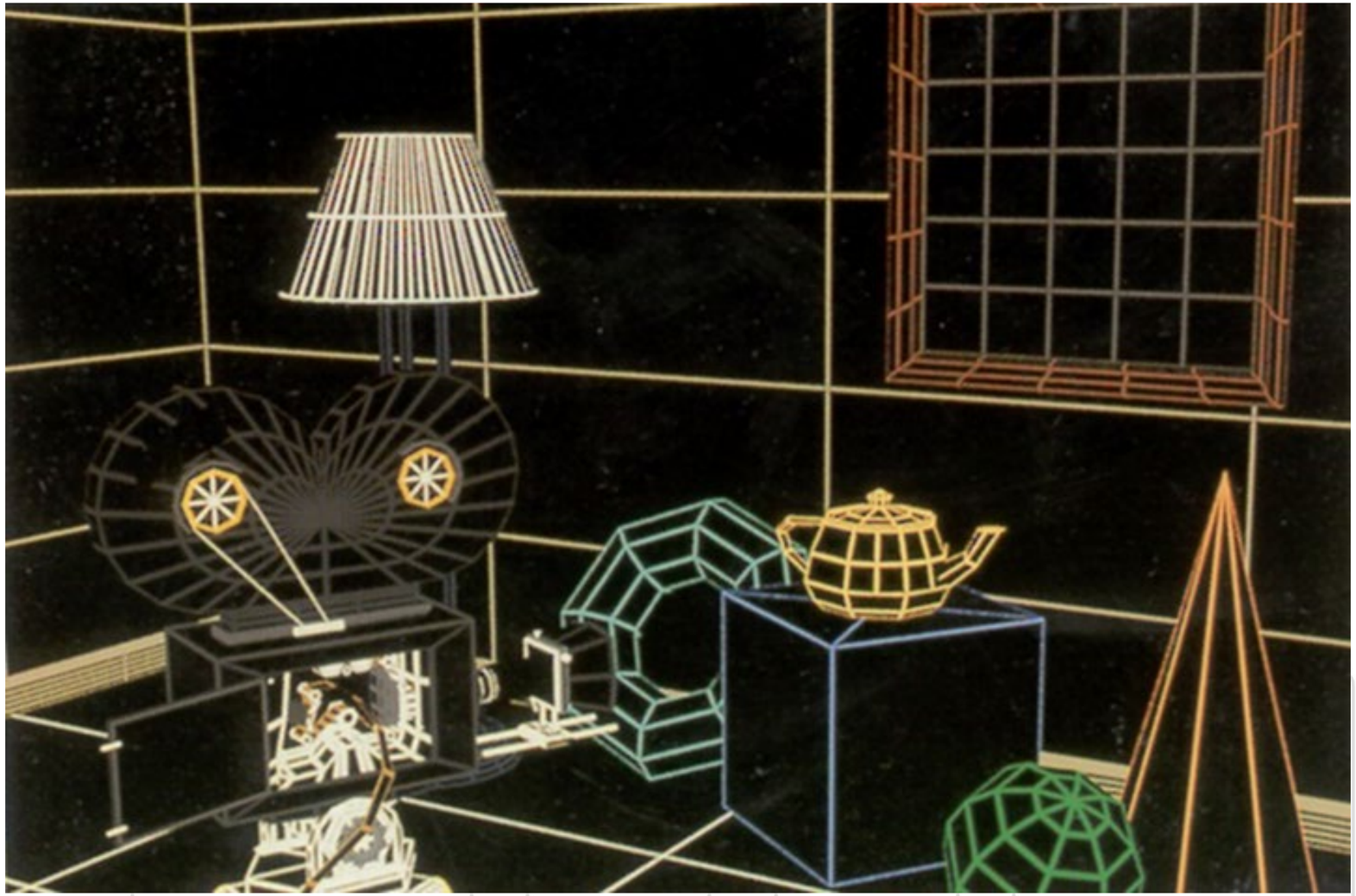
  - 其他变换
    - 无法通过渲染管线中的矩阵运算完成
    - 自由形变（freeform deformation）





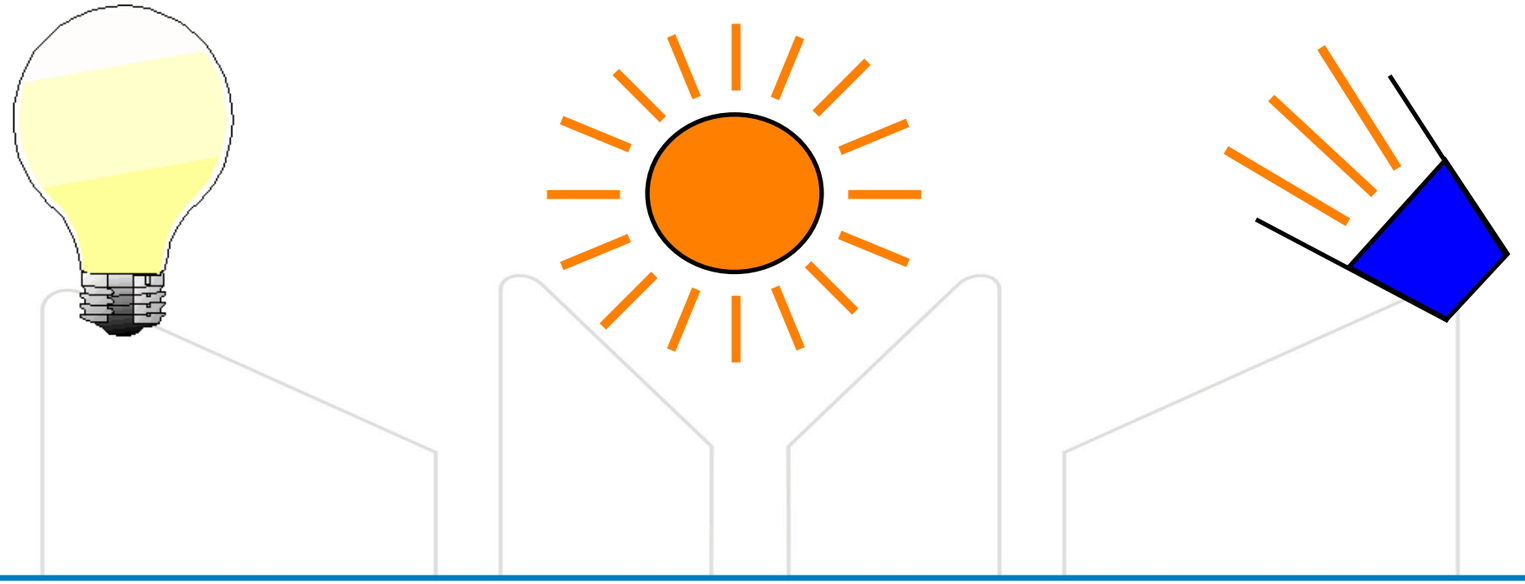
图形渲染管线

- 渲染举例：经过modelview变换得到的场景（无光照）



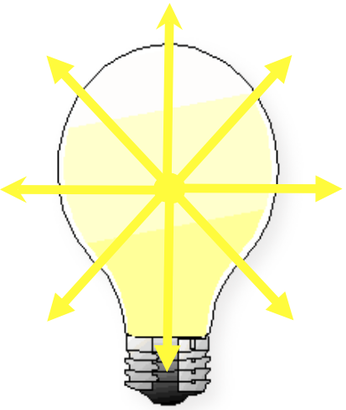
图形渲染管线

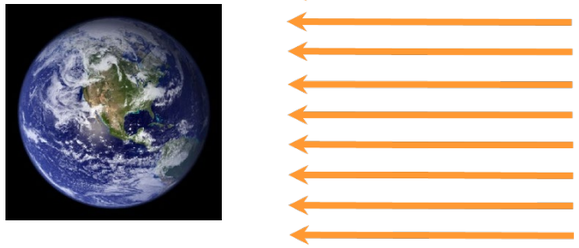
- 光照
  - 基于物体的材质和光源属性计算每个顶点的亮度
  - 现实中有多种光源
    - 灯泡、太阳、聚光灯、吸顶灯，等
    - 其发射光子（photons）的方式不同

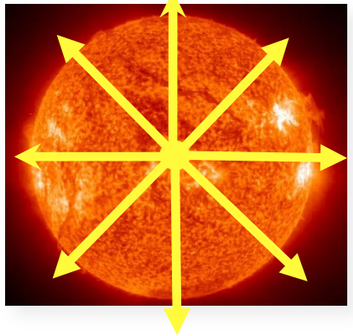


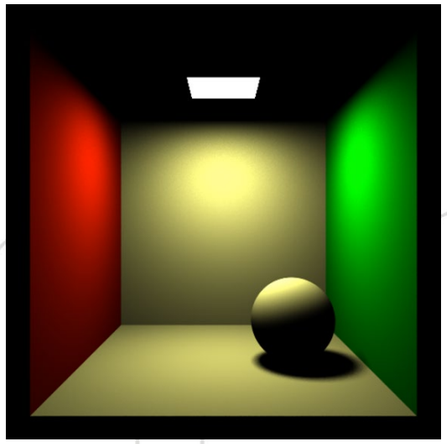
图形渲染管线

- 光照
  - 点光源（如，灯泡）
    - 向四面八方发射光
    - 所有光子从一个点出发，其发射方向均匀地覆盖整个球面
  - 方向光源（如，太阳）
    - 在无穷远处的一个点光源
    - 可以假设所有光束都平行
  - 区域光源（如，吸顶灯）
    - 在曲面上的每个点沿所有方向发光
    - 可想象为遍布整个曲面的点光源









图形渲染管线

- 渲染举例：加上光照的场景



图形渲染管线

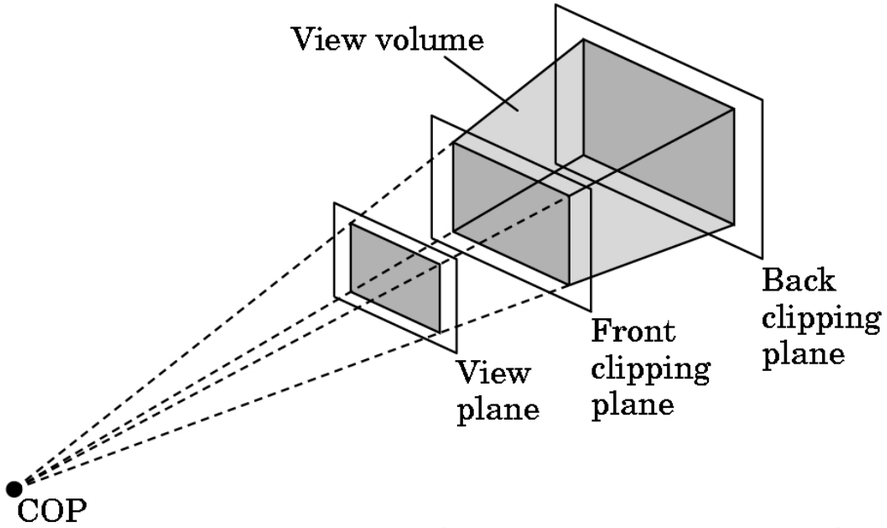
- 渲染举例：复杂明暗处理后的场景

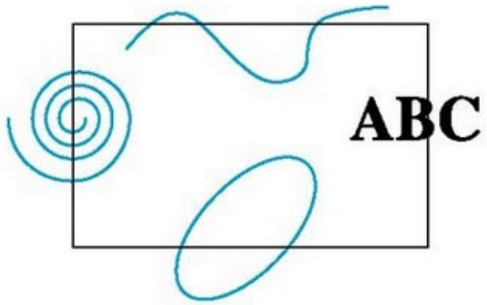


图形渲染管线

投影与裁剪

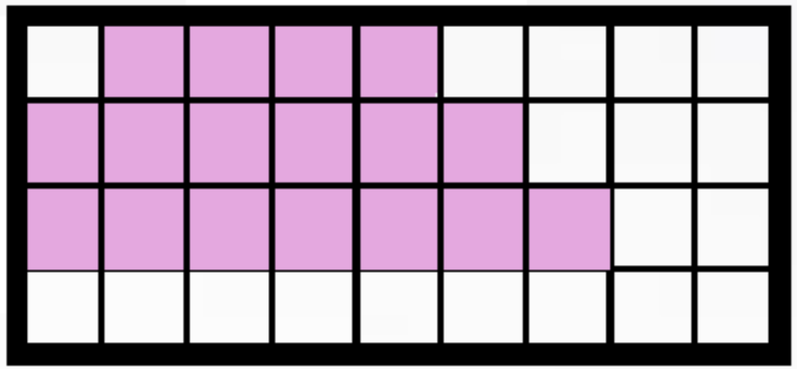
- 投影：3D到2D的变换
  - 可比照摄像机的工作方式，实际使用矩阵运算完成
- 裁剪（clipping）
  - 去除view volume外的内容





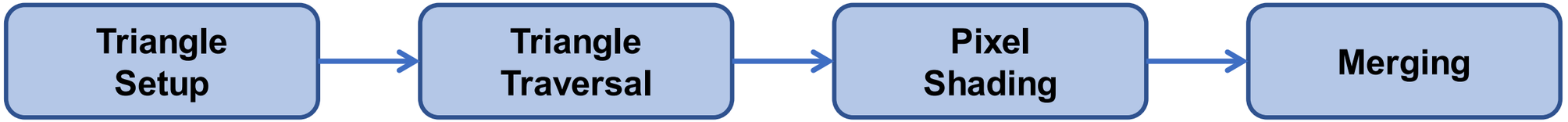


- 扫描变换
  - 将连续2D几何体（线、多边形，等）离散化
  - 光栅化显示（raster display）：离散网格

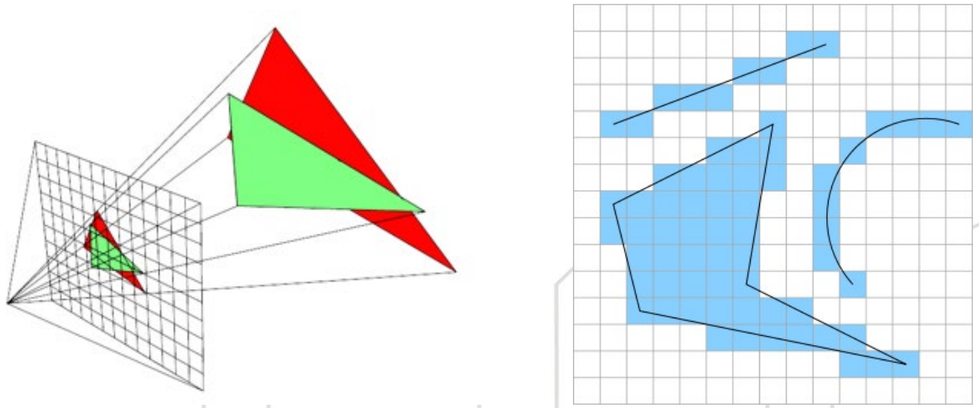


图形渲染管线

- 光栅（rasterizer）
  - 将屏幕空间中的二维顶点转换为像素
    - 考虑顶点的z-value（深度）及相关shading属性



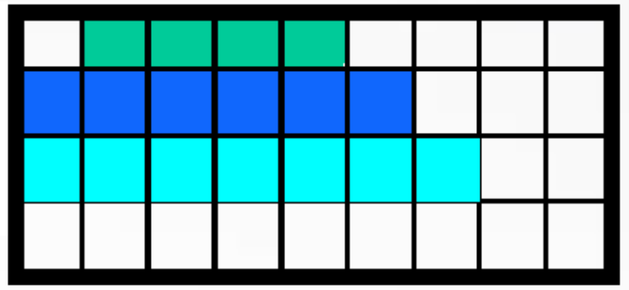
- 分辨率：转换后每行及每列像素的个数
- 屏幕空间：以像素为单位的二维直角坐标系

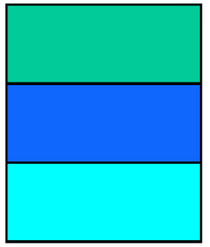


图形渲染管线

- 纹理映射（texture mapping）
  - “将图像贴在几何体上”
  - Fragment的颜色由图像中相应位置的颜色决定









Skin Image

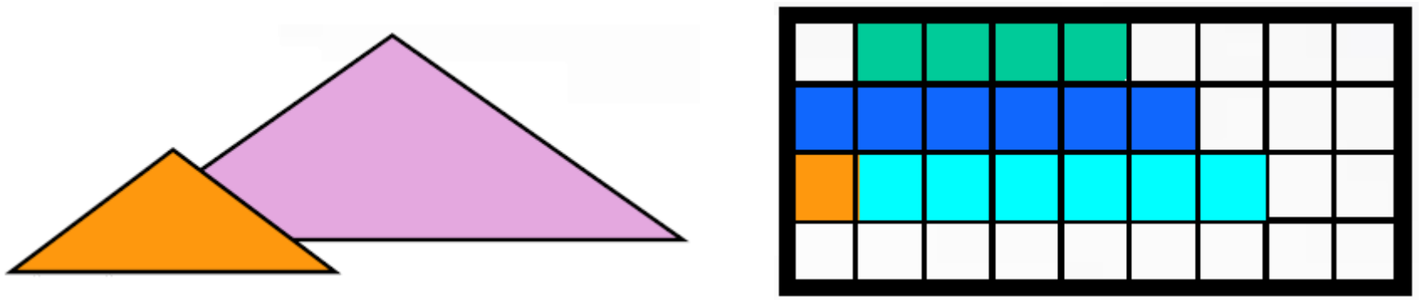
图形渲染管线

- 渲染举例：纹理映射后的场景



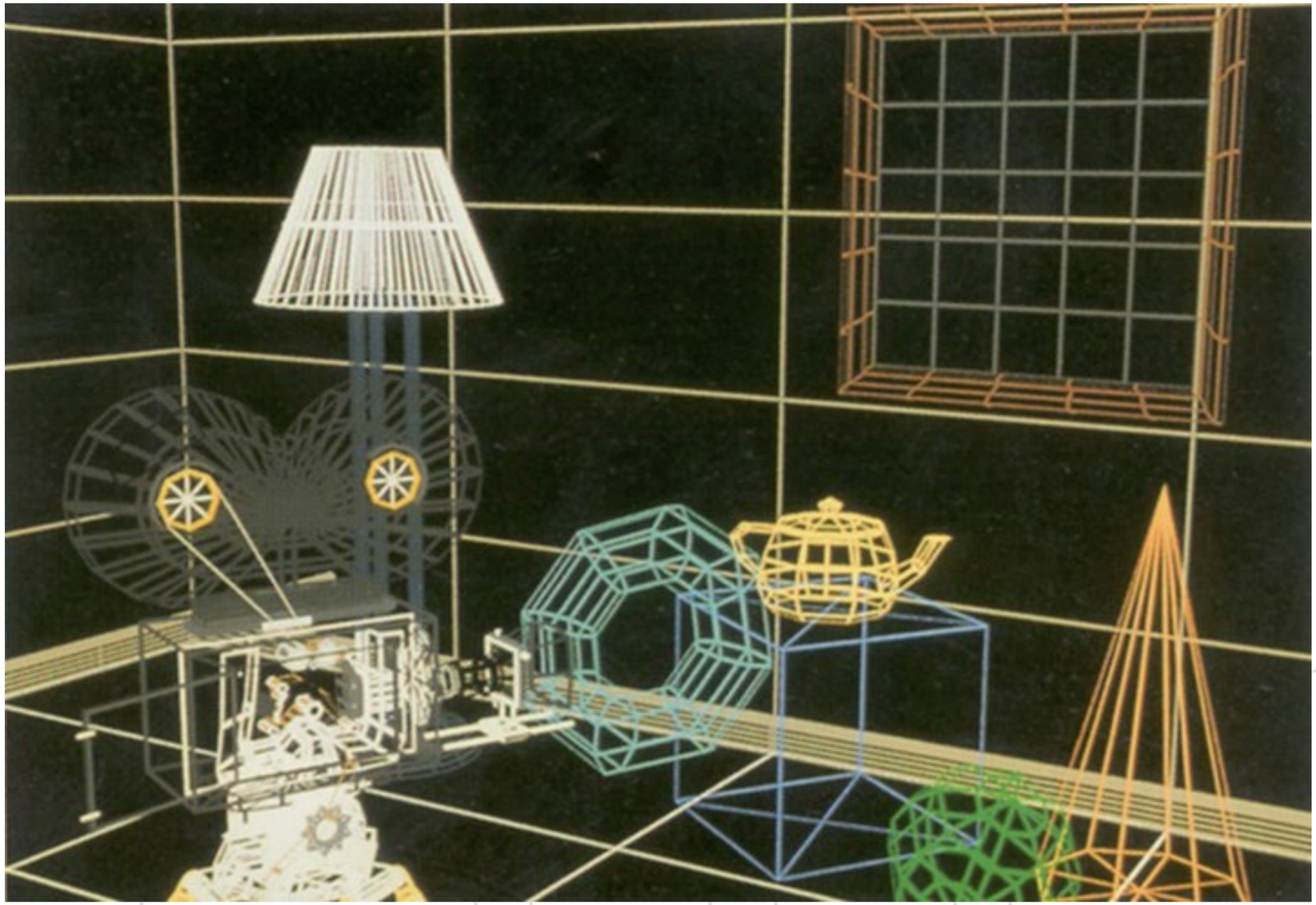
图形渲染管线

- 深度测试（depth test）
  - 将几何体被遮挡的部分从生成图像中去除
  - 对每个fragment进行测试



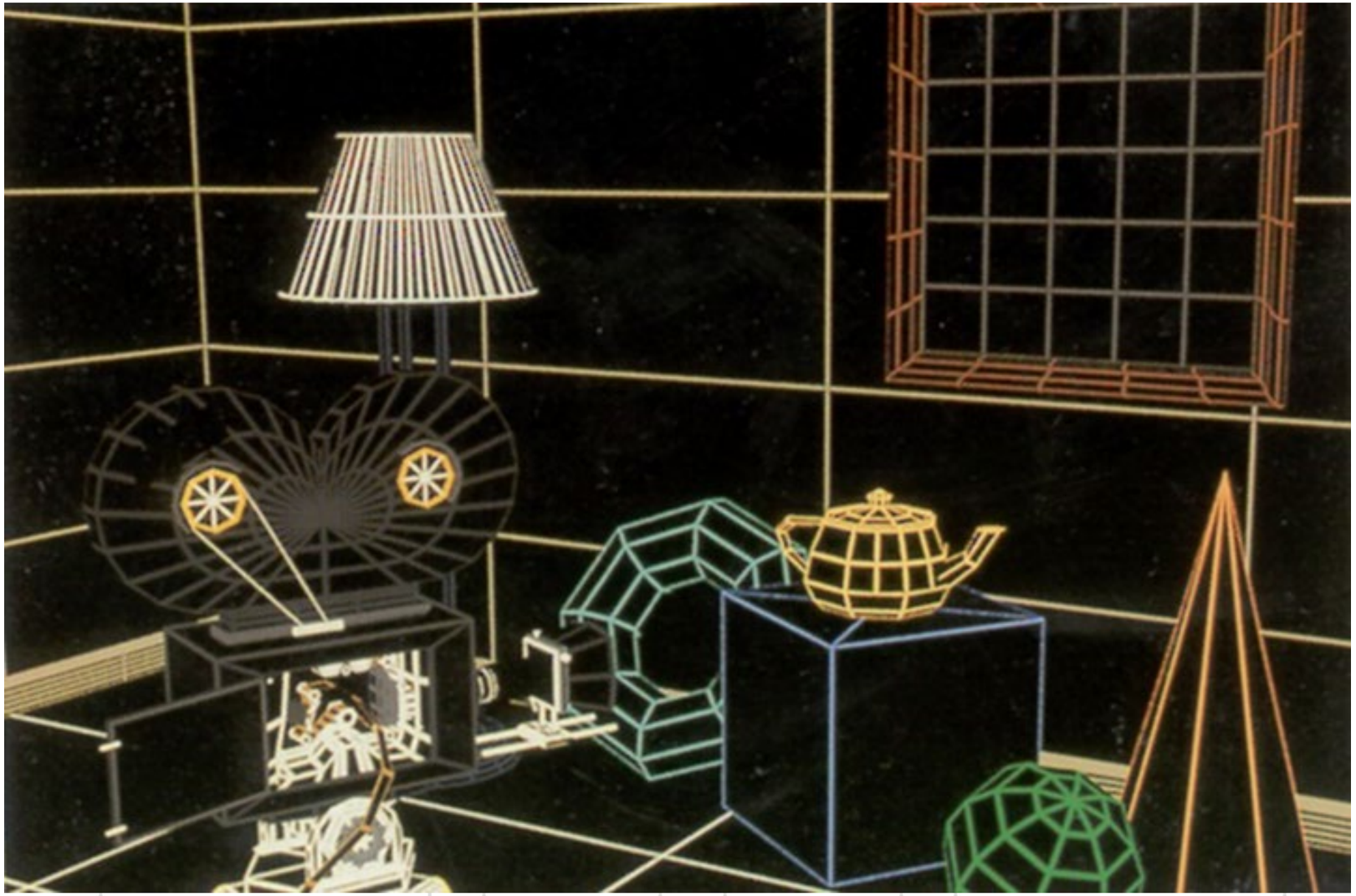
图形渲染管线

- 渲染举例：无消隐



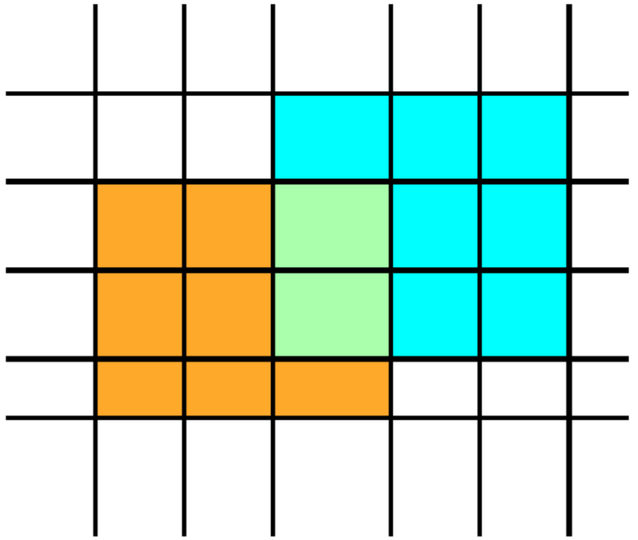
图形渲染管线

- 渲染举例：有消隐



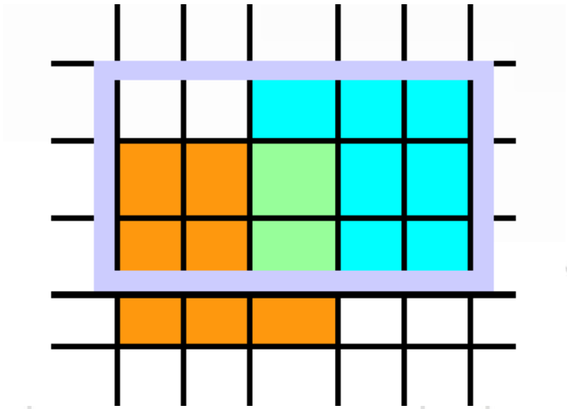
图形渲染管线

- Blending（颜色混合）
  - 最终生成图像：将fragment写至像素
  - 无blending：替换使用离屏幕较近的fragment替换较远fragment
  - blending：按最远fragment到最近的顺序（或相反）进行渲染
    - 顺序需要复杂算法保障（核心库不进行处理）
    - Porter-Duff合成（将颜色线性合成）



图形渲染管线

- Frame-buffer
  - 产生图像保存于显存中
  - 双缓存（double-buffering）
    - 两个独立缓存
    - 当显卡将结果绘制于其中一个缓存时，将另一个缓存的内容传至显示器进行显示，从而避免闪烁



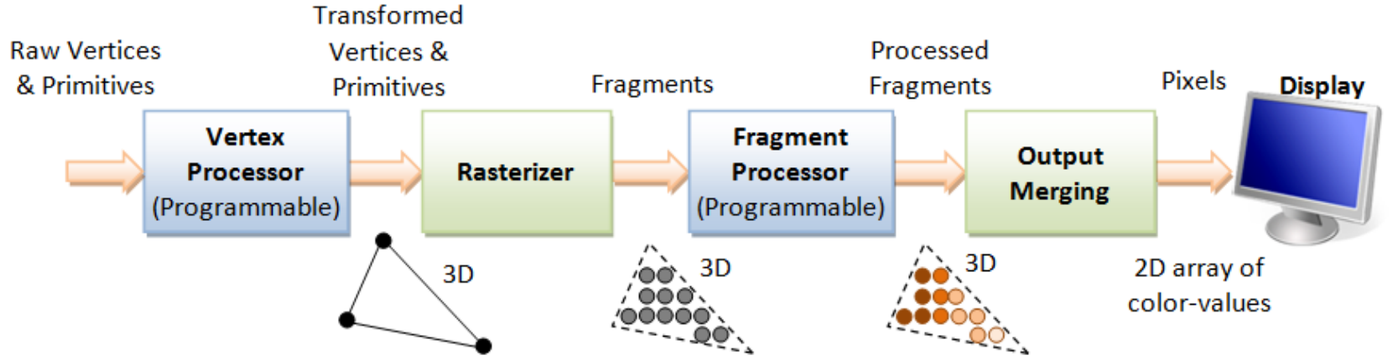
255 255 255	255 255 255	0 255 255	0 255 255	0 255 255
255 155 0	255 155 0	155 255 155	0 255 255	0 255 255
255 155 0	255 155 0	155 255 155	0 255 255	0 255 255

图形渲染管线

- 建模（Modeling） vs 渲染（Rendering）
  - 建模
    - 创建模型
    - 设置模型材质
    - 将模型置于场景中
    - 设置光源属性
    - 设置摄像机
  - 渲染
    - 使用虚拟相机“拍照”
    - 顶点到fragment
    - 决定fragment颜色
    - 决定pixel颜色

小结

- 渲染管线基本步骤
  - 顶点处理：对每个顶点进行独立变换
  - 光栅化：将由顶点定义的几何图源转成fragments
    - 每个fragment可以看做是按图像网格排列的三维空间中的一个像素
  - Fragment处理：对每个fragment进行计算（颜色等）
  - 合并：将fragment合并为像素



3D Graphics Rendering Pipeline: Output of one stage is fed as input of the next stage. A vertex has attributes such as (x, y, z) position, color (RGB or RGBA), vertex-normal (n_x, n_y, n_z), and texture. A primitive is made up of one or more vertices. The rasterizer raster-scans each primitive to produce a set of grid-aligned fragments, by interpolating the vertices.

Questions?

